

Select-Then-Compute: Encrypted Label Selection and Analytics over Distributed Datasets using FHE

Nirajan Koirala*, Seunghun Paik[†], Sam Martin*, Helena Berens*,
Tasha Januszewicz*, Jonathan Takeshita[‡], Jae Hong Seo[†], Taeho Jung*

*University of Notre Dame. Emails: nkoirala@nd.edu; smarti39@alumni.nd.edu; {hberens, ajanusze, tjung}@nd.edu

[†]Hanyang University. Emails: {whitesoonguh, jaehongseo}@hanyang.ac.kr

[‡]Old Dominion University. Email: jtakeshi@odu.edu

Abstract—Private Set Intersection (PSI) protocols allow a querier to determine whether an item exists in a dataset without revealing the query or exposing non-matching records. It has many applications in fraud detection, compliance monitoring, healthcare analytics, and secure collaboration across distributed data sources. In these cases, the results obtained through PSI can be sensitive and even require some kind of downstream computation on the associated data before the outcome is revealed to the querier, computation that may involve floating-point arithmetic, such as the inference of a machine learning model. Although many such protocols have been proposed, and some of them even enable secure queries over distributed encrypted sets, they fail to address the aforementioned real-world complexities.

In this work, we present the first *encrypted label selection and analytics* protocol construction, which allows the querier to securely retrieve not just the results of intersections among identifiers but also the outcomes of downstream functions on the data/label associated with the intersected identifiers. To achieve this, we construct a novel protocol based on an approximate CKKS fully homomorphic encryption that supports efficient label retrieval and downstream computations over real-valued data. In addition, we introduce several techniques to handle identifiers in large domains, e.g., 64 or 128 bits, while ensuring high precision for accurate downstream computations.

Finally, we implement and benchmark our protocol, compare it against state-of-the-art methods, and perform evaluation over real-world fraud datasets, demonstrating its scalability and efficiency in large-scale use case scenarios. Our results show up to $1.4\times$ to $6.8\times$ speedup over prior approaches and select and analyze encrypted labels over real-world datasets in under 65 sec., making our protocol practical for real-world deployments.

I. INTRODUCTION

Modern data-driven workflows frequently necessitate collaboration among multiple independent data custodians, each subject to stringent privacy laws and rigorous security regulations. Such collaboration is often only possible when both privacy and utility needs are met for all the involved parties. Across regulated domains, including finance, healthcare, and the public sector, organizations must balance the utility of collaborative analytics with statutory obligations to safeguard personally identifiable information (PII) [1], [2].

In the financial sector, institutions typically have limited visibility of their own customers' financial activities [3]. This fragmented view arises from the distribution of customer transactions across numerous entities, including other banks, credit card issuers, mortgage lenders, and mobile banking platforms. This fragmentation degrades fraud controls as models operate on partial views, inflating false positives and wasting analyst effort, while manual investigations often extend for weeks to months, delaying mitigation [4]. Collaboration that could close these gaps is impeded by onboarding friction, rising fraud pressure, and inter-institutional distrust, amplifying systemic risk. Privacy-preserving, cross-institutional analytics that assemble comprehensive risk profiles can reduce operational costs, mitigate financial risks, enhance profitability, and significantly accelerate investigations. Realizing this, however, demands strict compliance with financial and data-protection regulations [5], [6], [7] and credible mechanisms to resolve competitive sensitivities and institutional distrust. Similar challenges occur in healthcare, where sensitive electronic records, imaging, labs, and claims are siloed across hospitals, clinics, labs, and insurers; interoperability gaps and HIPAA constraints limit cross-institution visibility, impeding timely diagnosis, surveillance, and care coordination [8], [9], [10]. In real estate, where brokers, lenders, insurers, and listing platforms each hold sensitive financial and personal data, regulatory obligations (e.g., GDPR/CCPA) and competitive sensitivities discourage direct sharing, exacerbating fragmentation [7], [11], [12].

Moreover, such sensitive datasets can be massive, which intensifies the complexity of ensuring utility without compromising confidentiality. At scale, custodians delegate storage and compute to the cloud [13], [14] yet require designs that never expose plaintext to the provider. Regulations compel protection of customers' data [15], and providers likewise prefer encrypted delegation to limit breach liability [16], [17]. In sum, the widespread fragmentation of massive sensitive data across institutions highlights the critical need for accurate and scalable cross-custodian analytics that preserve data confidentiality and comply with regulatory requirements.

In many scenarios, these datasets include both PII identifiers and sensitive PII labels (or payloads) associated with them, and the capability to privately compute over such labels can unlock critical use cases [18]. For example, within anti-money-laundering (AML) efforts that can span thousands of banks

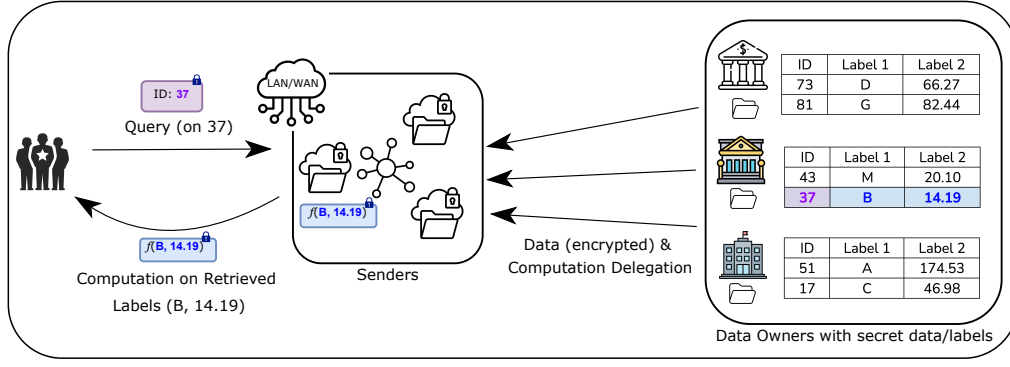


Fig. 1: Overview of encrypted label selection and analytics (ELSA) over-distributed (data, label) pairs. f designates a computation function (e.g., logistic regression, ML model) over the labels of the matching data. Senders and receiver are connected using a LAN or WAN.

[19], institutions must securely identify PII's such as account numbers and confidentially analyze associated financial data for risk scoring, all while preventing any disclosure of which institution provided the matching data [20], [21], [22], [23], [24]. Similarly, watchlist checks require agencies to securely query PII identifiers (passport numbers, national IDs) and privately retrieve and compute on related PII intelligence (e.g., arrest records, associates, travel histories) without revealing non-matching records [25], [26], [27]. Recent commercial offerings (e.g., Duality's Collaboration Hub with Oracle [28], [29]) signal strong market demand for such encrypted analytics over fragmented private financial datasets. In healthcare, providers or researchers require privately linking patient identifiers across hospitals, clinics, and laboratories and perform analytics on biomarkers (e.g., lab results) for disease-risk prediction and other diagnoses in compliance with interoperability rules [9]. To achieve robust privacy-preserving analytics across large-scale, data-fragmented environments, the aforementioned real-world examples highlight the need for a privacy-preserving protocol that satisfies the following (high-level) requirements.

- 1) **Label confidentiality.** Sensitive label vectors (e.g., transaction histories, credit data) linked to identifiers must remain confidential end-to-end during selection and analytics.
- 2) **Privacy-preserving analytics.** Modern risk rating increasingly relies on statistical and ML models [30]; the system must support complex computations (e.g., inference, risk scoring) on PII's such as associated labels to the identifiers without compromising privacy.
- 3) **Data fragmentation.** High-dimensional identifier-label records are distributed across many custodians under independent governance; secure federated selection and computation must proceed without centralizing data and scale with both the number of parties and data volume.

Querying large-scale, geographically distributed datasets is computationally intensive but feasible in non-private settings [31], [32]. Introducing strict privacy requirements, however, significantly increases both the computational and communication overheads. In the canonical workflow, a querier starts with a single unique identifier, say y (e.g., a bank-A/C number, passport ID, etc.) and must answer one compound question:

- 1) Selection: Test whether any custodian holds identifier y .

- 2) Label analytics: On a hit, select labels(y) and compute $f(\text{labels}(y))$; otherwise, abort.

The two-stage pipeline separates selection from computation and executes expensive private analytics only when a positive match occurs. Existing primitives, such as private set intersection (PSI), labeled-PSI, circuit-PSI, private information retrieval (PIR), or their variants, fundamentally stop at the selection stage. They either reveal the intersection itself or nothing at all, support only restricted function classes (often symmetric) and data types, and cannot advance to encrypted analytics without leaking non-matches or label values. Many of these approaches also lack native support for real-valued payloads required by ML inference. Thus, executing both stages, selection and label analytics, while adhering to the requirements mentioned earlier, while ensuring low computational and communication costs, lies beyond current privacy-preserving methods. These requirements jointly give rise to a multi-institution setting in which many custodians hold encrypted (identifier, label) records and the goal is to securely compute, across parties, an arbitrary function on the labels (potentially real-valued) for a queried identifier, if the identifier is present, while revealing nothing about non-matches or intermediate values and disclosing only the final output to an authorized recipient. We term this functionality as encrypted label selection and analytics (ELSA).

We illustrate the ELSA functionality to be realized in this work with an AML use case in Figure 1. An investigator encrypts a PII account identifier (e.g., ID 37) and requests a risk score computed over sensitive label vectors held across multiple institutions. Each institution delegates an encrypted (identifier, label) table to the cloud under a common FHE public key; label vectors encode AML features per the FFIEC BSA/AML manual [33], [34] (e.g., transaction anomalies, regulatory reports). The protocol (i) privately selects the identifier across institutions, (ii) homomorphically computes the risk function on matched labels, and (iii) returns only the encrypted score to the investigator, revealing neither raw labels nor any information about non-matches.

Limitations of existing methods. There have been several variants of PSI protocols to support label retrieval or computation over labels. For example, labeled PSI [35], [36], [37],

[38] allows the *receiver* to learn the associated data (labels) of the matched item held by the *sender*. Circuit-based PSI [39], [40], [41], [42], [43], [44], [45], [46] and their variants, e.g., private matching for compute (PMC) [47], [48] or private join and compute (PJC) [49], [50], allows computations on the intersected data, such as cardinality of matched items or summation of associated data for matched items.

However, these protocols have various limitations in achieving the requirements of ELSA functionality. Labeled PSI protocols release the plain label to the receiver; hence, they do not support computation over the labels without revealing them. Although circuit-based PSI protocols support generic computations through secure multi-party computation (SMPC) techniques, these methods require multiple rounds of substantial communication between the receiver and the sender. In particular, their methods do not natively support real-valued arithmetic, so complex real-valued computations, e.g., ML inference, may incur significant computational and circuit complexity overheads. Alongside these limitations, all these works assume that the ID and label pairs are stored in plaintext, and are only designed for two-party scenarios; extending these protocols for a large number of servers with encrypted data is non-trivial. Some recent studies, e.g., Koirala et al. [51] and Mouris et al. [48], attempted to consider PSI or PMC over encrypted data. Although these protocols scale up to thousands or more parties, the former supports neither retrieving labels nor downstream computation, and in the latter, their protocol supports only limited types of computations, e.g., left join.

Our Solution. In this work, we propose a protocol for achieving the ELSA functionality that addresses multiple shortcomings of previous works. Our protocol supports arbitrary computations on the (encrypted) labels resulting from matched records, which are distributed across a large number of senders, e.g., thousands or more. Unlike methods where downstream computations are supported by only SMPC techniques, we require no computation for the receiver, and all the heavy computations are offloaded entirely to the resource-rich senders. Our protocol natively supports real-valued downstream computations, enabling rich analytics on labels derived from matched identifiers. Since these computations run under FHE across multiple custodians, key management is a major concern. For clarity, we initially assume a trusted third party (TTP) that provisions key management and distribution, and later remove this assumption by instantiating a threshold CKKS [52] scheme. Threshold-FHE is orthogonal to our design, and any type of threshold CKKS FHE is compatible with our protocol design. We introduce two major novel techniques: 1) generalization of the domain extension polynomial by Cheon *et al.* [53], and (2) slot-wise windowing technique. These two techniques enable our protocol to homomorphically evaluate the equality (or *if*) function over large domains with high accuracy and low FHE depth to be able to handle billions of identifier-label pairs for achieving ELSA. Based on these technical novelties, along with several other optimizations, we build a protocol for ELSA that is secure under a semi-honest adversary model. The contributions of this work are

summarized as follows:

- We design the first CKKS-based protocol for ELSA that supports real-valued downstream computations without requiring SMPC or receiver-side heavy computations.
- We propose a novel approximation for homomorphic equality testing over large domains (2^{64} to 2^{128}), which can be efficiently evaluated while operating over small FHE presets.
- We propose several improvements and optimizations on the building blocks of our approximation method, including domain extension polynomials (Cheon *et al.* [53]) and slot-wise windowing, which sharply reduces the unit cost for the selection stage while ensuring high precision after downstream computations.
- We implement our protocol and evaluate it on real-world datasets, demonstrating end-to-end function evaluation in under 65 sec. In the selection setting, we achieve up to $1.4\times$ to $6.8\times$ speed-ups over prior state-of-the-art methods. Our anonymized source code is available at <https://anonymous.4open.science/r/ELSA-4242> for future reproducibility.

II. RELATED WORK

A. Private Set Intersection (PSI)

1) *Circuit-PSI*: Huang *et al.* [39] initially extended PSI to circuit-PSI, which enables parties with private input sets X and Y to privately compute any *symmetric* function over the intersection $X \cap Y$ via secure evaluation of a boolean circuit [58], [40]. This computation allows symmetric functions such as cardinality, sum over associated attributes, or threshold intersection, which have many applications [59], [60], [61]. CHLR18 [35] extends FHE-based PSI to a labeled setting and describes how labels obtained after PSI can be masked and fed into a downstream generic SMPC. However, they mostly describe circuit-PSI over the labels in theoretical terms without any experimental validation. Additionally, the SMPC methods proposed in their work would induce a large communication overhead. Son *et al.* [41] built a circuit-PSI protocol based on the works of [35], [36], but their work only supports a few functions over the intersection and does not operate on labels.

Mahdavi *et al.* [37] proposed PEPSI, where the client sends an encrypted set, and the server computes an arbitrary function over the intersection. Their computation operates on the intersecting elements rather than their associated labels. Other works, such as [62], [42], [43], [44], [45], [63], [46], extend circuit-PSI to circuits on data associated with the intersection. However, the functions they compute are aggregations or cardinality, which are small circuits (nearly linear in the set size) to limit overheads. These constructions could be used for general post-intersection computation, but using them for more complex functions with floating-point operations can be very costly in terms of gate count and communication overhead.

2) *Multi-Party PSI (MPSI)*: Kissner and Song [64] proposed one of the first works to yield intersection, union, and threshold for MPSI, and others like Nevo *et al.* [55] built malicious MPSI protocols based on oblivious programmable PRF (OPPRF) and oblivious key-value store (OKVS). Chandran *et*

TABLE I: Comparison with related works. Notation: L.R and L.P denote Label Retrieval and Label Privacy, respectively. Real-Valued Func. denotes computational capability on input labels that are real numbers. — denotes not applicable. VAF and wDEP defined in Sections VI-A.

Protocol	Building Blocks	L.R	L.P	Computation over Label	Real-Valued Func.	Multi. Sender Scale	Security Model
Ion <i>et al.</i> [50]	OT + Hashing	✓	✓	Not supported	—	2-party	Semi-honest
Lepoint <i>et al.</i> [49]	OPRF + PIR + HE	✓	✓	Inner-product PJC only	—	2-party	Semi-honest
Cong <i>et al.</i> [36]	FHE + OPRF	✓	×	Supported via MPC*	×	2-party	Malicious
Rindal <i>et al.</i> [46]	OT (Vector-OLE) + Garbled circuit	✓	✓	Supported via MPC*	×	2-party	Semi-honest (opt. malicious)
Mahdavi <i>et al.</i> [37]	FHE	✓	×	Supported via FHE	×	2-party	Semi-honest
Son <i>et al.</i> [41]	FHE + OPRF	✓	×	Supported via MPC*	×	2-party	Semi-honest
Mouris <i>et al.</i> [48]	Rerandomizable Encrypted OPRF	✓	✓†	Left-Join PMC only	—	1000s or more	Semi-honest
Koirala <i>et al.</i> [51]	Threshold FHE	×	—	—	—	1000s or more	Semi-honest
Chandran <i>et al.</i> [54]	OPPRF + MPC (garbled circuits)	×	—	—	—	Up to 15 parties	Semi-honest
Nevo <i>et al.</i> [55]	OPPRF + OKVS	×	—	—	—	Up to 32 parties	Malicious
Vadapalli <i>et al.</i> [56]	DPF + ORAM	×	—	—	—	3-party	Semi-honest
Vos <i>et al.</i> [57]	EC-ElGamal + OKVS	×	—	—	—	Up to 100s	Semi-honest
This work	VAF, wDEP, Threshold FHE	✓	✓	Supported via FHE	✓	1000s or more	Semi-honest

*MPC requires a higher number of rounds of communication, as compared to FHE.

†Intersection sizes leaked to the delegator party (sender).

al. [54] proposed multiparty circuit and quorum PSI protocols, which hide the intersection but evaluate a function on that intersection and identify which elements in a designated party's set appear in at least k other parties' sets. Other works such as [65], [66], [67], [68] have built multi-party private set union protocols, which is a problem setup different from ours.

3) *Delegated PSI*: In delegated PSI, each user, including the querying user, securely delegates their set to the cloud server, and ultimately, the querying user learns the intersection result. Parties may not trust the cloud, which makes it very similar to this work's scenario. Kerschbaum *et al.* [69] proposed one of the earliest PSI protocols in which computation of the intersection is outsourced to an oblivious service provider. Duong *et al.* [70] presented PSI protocols to compute the cardinality of intersections for privacy-preserving contact tracing, and other works such as [71], [72] also operate under similar settings. Despite the efficiency and multi-party support in these works, there are still limitations for scenarios requiring long-term storage and queries over fully encrypted data. These works are specialized for only limited functions, similar to works under circuit-PSI, and their security crucially depends on no client colluding with the server or other clients, and they do not scale easily in terms of multiple delegated cloud servers.

B. Private Membership Test (PMT)

The private membership test (PMT) is a cryptographic primitive that allows a receiver to learn whether its singleton input y is contained in a large dataset X held by a sender. Recently, Garg *et al.* [73] have proposed a 2-round PMT protocol for string equality based on the DDH or LWE assumptions. [74] and [75] have also built PMT protocols based on the BFV FHE scheme and cuckoo filters, respectively. Other works such as [76], [77], [78], [68] have also proposed various PMT protocols in the two or multi-party settings but operate over limited privacy settings for the datasets they operate on. Koirala *et al.* [51] recently proposed a threshold-FHE-based private segmented membership test (PSMT) protocol that aggregates responses from many servers in a single homomorphic summation. However, their work is limited to a yes/no membership result and does not retrieve any associated labels of the matching results from the server(s).

C. Other Similar Methods

Some works have focused on performing a downstream computation on the matched data after the match has been established privately. Ion *et al.* [50] proposed PSI with cardinality over the matched data for aggregating ad conversion rates. Lepoint *et al.* [49] defined the private join and compute (PJC) functionality that enables secure computation over data distributed across different databases. They expanded on [50] by enabling computation over the intersection but keeping the intersection itself hidden using Private Information Retrieval (PIR) and differential privacy techniques. Recently, Chida *et al.* [79] proposed a more communication- and round-efficient construction than [49]. However, these works are tailored towards computing a dot product of intersecting items and do not support arbitrary computations. Furthermore, they do not naturally extend to multi-party scenarios, and one would have to orchestrate multiple pairwise PJC computations or integrate them into an n -party SMPC [17].

Another line of work called private matching for compute (PMC) [47] enables aggregate downstream computation on the intersection. [48] and [80] improved upon the previous works by enabling private record linkage in a delegated setting and collaborative data analysis. One major limitation of these works is that they mainly focus on simple computations, like aggregation, and require additional SMPC for complex computations. Duoram [56] uses distributed oblivious RAM for private record retrieval but does not support post-join computations. Vos *et al.* [57] address repeated membership queries in cross-silo federations but reveal matching parties and only return a plaintext yes/no flag.

We compare representative prior works with ours in Table I.

III. PRELIMINARIES

In this section, we summarize some of the important notations used in this work. y is the receiver's singleton query and sets X_i and ℓb_i are data owner's sets. δ denotes the length of items in y , X_i and ℓb_i . κ is the slot-wise windowing parameter. n , N , D , η denote the number of senders, FHE ring dimension, FHE depth, and batch size, respectively. $select_i$, $resLab_i$, $perm_i$, $flag_i$ and $c_{\ell b_{stat}}$ denote the output of the selection stage, ciphertext containing labels, ciphertext containing 1 in slot ρ_i , intersection indicator ciphertext and

statistics ciphertext respectively for the i -th sender such that $i \in \{1, \dots, n\}$.

(Leveled) Fully Homomorphic Encryption: Fully Homomorphic Encryption (FHE) schemes allow computation on encrypted data. The Learning With Errors (LWE) problem and its variant Ring LWE (RLWE) underpin the security of modern FHE schemes [81], [82]. Since the first theoretical realization of FHE in 2009 [83], much work has been done to make FHE more practically usable. The most prominent FHE schemes are BGV [84], B/FV [85], [86], CKKS [52], and TFHE [87]. For improved performance, the encryption parameters for FHE schemes are typically chosen to support only circuits of a certain bounded multiplicative depth (leveled FHE), which we use in our implementation. Our work uses the CKKS scheme as we deal with elements and associated data that are typically represented in floating-point arithmetic and can be used for analytics via ML, logistic regression, etc. Similar to B/FV and BGV, CKKS operates upon $\mathcal{R} = \mathbb{Z}[X]/\langle \Phi_M(X) \rangle$, where $\Phi_M(X)$ is the cyclotomic polynomial $(x^N + 1)$ of order $M = 2N$ (cyclotomic index) and degree $N \in \mathbb{Z}$ called the ring dimension. CKKS parameters include the ring dimension, ciphertext modulus q , scaling factor Δ , and the standard deviation of the error. In FHE schemes (except TFHE), SIMD (Single Instruction Multiple Data) encoding of plaintexts is possible, greatly increasing throughput. CKKS allows encrypting $\frac{N}{2}$ complex values into a single ciphertext.

Threshold FHE: For threshold functionality, we use an α -out-of- n threshold-FHE (thresFHE) scheme, where decryption requires α parties [88]. A thresFHE scheme consists of a tuple of probabilistic polynomial time (PPT) algorithms (*Enc*, *Eval*, *PartialDec*), and two n -party protocols (*KeyGen*, *Combine*) with the following functionalities:

- **KeyGen**($1^\lambda, 1^D, params$) $\rightarrow (pk, evk, \{sk_i\}_{i \in [n]})$: Given a security parameter λ and a depth D , each party P_i outputs a common public key pk for encryption, a common evaluation key evk , and a secret key share sk_i of the implicitly defined secret key sk under some public parameter $params$.
- **Enc**(pk, m) $\rightarrow c$: Given a public key pk and a message m , the encryption algorithm outputs a ciphertext c .
- **Eval**($evk, f, \{c_i\}_{i \in [v]}$) $\rightarrow c^*$: Given an evaluation key evk , a v -input function f that can be evaluated using at most depth D and ciphertexts $\{c_i\}_{i=1}^v$, the evaluation algorithm outputs a new ciphertext c^* which is an encryption of $f(m_1, \dots, m_v)$, where $c_i \leftarrow \text{Enc}(pk, m_i)$.
- **PartialDec**(c, sk_i): Given a ciphertext c and a secret key share sk_i , the partial decryption algorithm outputs a partial decryption result $pdec_i$.
- **Combine**($pk, \{pdec_i\}_{i \in [I]}$) $\rightarrow m$ or $\perp$: Given a public key pk and a set of partial decryptions $\{pdec_i\}_{i \in [I]}$ for an index set $I \subseteq [n]$, the combine algorithm returns the decryption result m if $|I| \geq \alpha$ otherwise $\perp$.

During the partial decryption, a smudging noise from some predefined distribution is added to hide the secret key shares and the RLWE error accumulated from homomorphic operations [89], [88]. The key generation in thresFHE can be accomplished either using a trusted setup procedure to distribute

partial secret keys, which can be executed via methods such as trusted hardware, or SMPC [89], [90]. We assume that the decryption threshold $\alpha < n/2$, i.e., honest majority setting. We require that a thresFHE scheme satisfy standard notions of compactness, correctness, and security [91].

Value Annihilating Functions (VAF): We use value annihilating function (VAFs) [51] as a building block for doing label selection and retrieval. It was defined by Koirala et al. [51] and informally, for a bounded domain $[-M, M]$, we say that the function f is a VAF if it satisfies $f(0) = 1$ and $f(x) \approx 0$ otherwise. From this property, we can observe that for $y \in [-M, M]$ and $X \subset [-M, M]$, $y \in X$ if and only if $\sum_{x \in X} f(y - x) > 0$, i.e., checking whether the additive aggregation of VAF evaluation results is zero or not.

IV. ENCRYPTED LABEL SELECTION AND ANALYTICS

In this section, we introduce our main functionality, encrypted label selection and analytics (ELSA), which performs the following: (1) selects the encrypted labels exclusively over encrypted and distributed identifier-label pairs and (2) processes downstream computations over the associated encrypted labels of the identifier.

Problem Formulation: We consider three types of entities: *data owner*, *sender*, and *receiver*. The data owner handles a dataset that consists of pairs $(x_i, lb_i) \in \mathcal{I} \times \mathcal{L}$, where $\mathcal{I}$ and $\mathcal{L}$ denote the universe sets for the identifiers and labels, respectively. We assume the data owner who, either due to resource constraints or operational preferences, delegates the encrypted database to a designated sender rather than participating directly. This setup parallels delegated PSI, freeing the owner from storing data locally or staying online [92].

Each sender is assumed to be a cloud server with strong computational and storage capabilities. They hold a pair of ciphertexts, one encrypting the unique identifiers and the other encrypting the corresponding labels. Following standard assumptions [93], [94], we assume the ordering of identifiers matches the ordering of labels. We designate one of the senders as the *leader sender* at the setup phase. The leader sender is responsible for evaluating a pre-defined function f , which takes encrypted labels held by each sender as inputs.

The receiver has access to a unique identifier as a query and learns the evaluation result of f on the labels of the matched elements. If a sender does not hold a matched identifier, it can result in the corresponding label becoming a “null” denoted as $\perp$, and the sender’s labels would not contribute to the function computation. For this reason, we let the receiver learn separately whether the query is in the sender’s set or not.

Function Evaluation: Our definition covers several types of functions analogous to existing circuit-PSI protocols. For instance, we can implement PSI with cardinality (f_{CA}) [45], which returns the number of matched items, or quorum PSI ($f_{QR, \tau}$) [54] with a threshold τ , which returns a predicate bit indicating whether the number of matched items surpasses τ or not. If we set $\mathcal{L} = \{0, 1\}$, $lb_i = 1$ for all identifiers in

Parameters: The protocol involves n parties: $S_1, \dots, S_{n-1}$ senders and a single receiver $\mathcal{R}$. We denote the dataset outsourced to each S_i as $\mathcal{X}_i : i \in [n-1]$. Sets $\mathcal{I}$ and $\mathcal{L}$ contain the identifiers and corresponding labels, respectively. A function $f : \prod_{i=1}^{n-1} (\mathcal{L} \cup \{\perp\}) \rightarrow \mathbb{R}^m$ for $m \in \mathbb{N}$.

Inputs:

- $\mathcal{R}$: An identifier $y \in \mathcal{I}$.
- S_i : An encryption of a set $\mathcal{X}_i \subset \mathcal{I} \times \mathcal{L}$.

Outputs:

- $\mathcal{R}$: Function evaluation result $f(\ell_{b_1}, \dots, \ell_{b_{l-1}})$ and the flag bit $b \in \{0, 1\}$, where ℓ_{b_i} is the associated data corresponding to $y = x_i$. If there is no matched identifier for the i 'th sender, then $\ell_{b_i} = \perp$. In addition, $b = 1$ indicates $y \in \bigcup_i \mathcal{X}_i$ and $b = 0$ otherwise.
- S_i : Outputs $\perp$.

Fig. 2: Ideal functionality of ELSA

$\bigcup_{i=1}^{l-1} \mathcal{X}_i$, and $\perp = 0$, then the following circuits implements each functionality, respectively.

$$f_{CA}(L) = \sum_{i=1}^{l-1} \ell_{b_i}, \quad f_{QR, \tau}(L) = 1 \left(\sum_{i=1}^{l-1} \ell_{b_i} > \tau \right)$$

where $L = (\ell_{b_1}, \dots, \ell_{b_{l-1}})$, which contains all the labels required to compute the respective functions. Similarly, we can also implement several functionalities of PMC [47] or PJC [49], such as average or (weighted) summation on the associated labels.

We emphasize that our notion of analytics on encrypted labels is much different from the standard circuit-PSI approaches that compute functions strictly on intersection sets. Traditional circuit-PSI works (e.g., [41], [45], [65], [40], [37]) focus on symmetric functions over intersected items, which do not align directly with our two-phase setting. In contrast, our approach enables computation on matched real-valued labels, i.e., $\mathcal{L} = \mathbb{R}$, reflecting practical use cases (see Section I).

We state the ideal functionality for ELSA in Figure 2.

Security Model: We provide a security definition in the presence of a semi-honest adversary. We assume a semi-honest setting in which all parties act honestly but remain curious. To realize threshold functionality in the full protocol, we use α -out-of- n thresFHE CKKS scheme where $\alpha < n/2$.

We assume adversaries, potentially corrupting up to $\alpha - 1$ parties, aim to learn either the receiver's query y (unless the receiver itself is compromised) or any sender's set element $x \in X_i$ and label $\ell \in \ell_{b_i}$. To show the security in the presence of a semi-honest adversary, we need to show an efficient simulator that can simulate all the *views* of the corrupted parties, i.e., received communications during the protocol, through the inputs and outputs of the protocol. We provide the formal definition as follows:

Definition 1. Let $\mathcal{F}$ be a n -party functionality and Π be a n -party protocol for parties $S_1, \dots, S_n$. We say that Π securely implements $\mathcal{F}$ at the presence of semi-honest adversaries with at most $(\alpha - 1)$ collusions if for every PPT adversary $\mathcal{A}$ controlling at most $(\alpha - 1)$ colluding parties among $S_1, \dots, S_n$, there exists a simulator $\mathcal{P}$ such that for all PPT distinguishers $\mathcal{D}$,

$$|\Pr[\mathcal{D}(\text{VIEW}_{\text{real}}) = 1] - \Pr[\mathcal{D}(\text{VIEW}_{\mathcal{P}}) = 1]| \leq \text{negl}(\lambda)$$

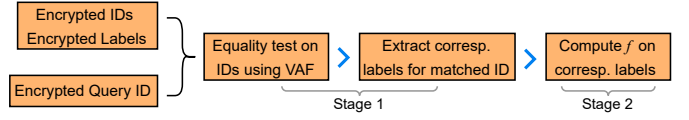


Fig. 3: Core components of ELSA protocol

where $\text{VIEW}_{\text{real}}$ is the view of the colluding parties during a real protocol execution and $\text{VIEW}_{\mathcal{P}}$ is the simulated view.

V. STRAWMAN PROTOCOL AND ITS CHALLENGES

We instantiate our basic protocol as a strawman protocol. The core components of the strawman design are displayed in Figure 3. We temporarily assume a trusted third party (TTP) distributes the encryption, evaluation, and decryption keys to data owners, senders, and the receiver, respectively (this assumption will be removed in Section VII by introducing thresFHE). Each sender receives encrypted tuples of unique IDs and associated labels, $(c_{x_i}, c_{\ell_{b_i}})$ from data owners via a secure channel. c_{x_i} is a encryption of $x_i \in \mathbb{Z}$ (or hashed δ -bit string), and $c_{\ell_{b_i}}$ is its associated encrypted label. If a sender's set is larger than the batch capacity η , it stores multiple FHE ciphertexts. Senders are also equipped with a function $f(\{\ell_{b_i}\}_{i \in L}; \Theta) = z$, where Θ can be a private value (e.g., ML model's private parameters [95]). The receiver holds a single query element $y \in \mathbb{R}$, which it replicates into all slots of the ciphertext c_y . The strawman protocol has two stages:

Stage 1 (Label Selection and Extraction): Upon receiving c_y from receiver, each sender S_i homomorphically,

- 1) Computes the difference of the query ciphertext c_y with its ciphertext c_{x_i} component-wise to obtain diff_i .
- 2) Applies a piecewise transform, value annihilating function (VAF) over diff_i that outputs 1 in each slot if $(y = x_i)$ for that slot, and 0 otherwise to obtain select_i .
- 3) Multiplies select_i with $c_{\ell_{b_i}}$ to obtain resLab_i . If multiple set encryptions exist, the sender sums all resulting resLab_i values into a single resLab_i .
- 4) Transmits its resLab_i to a designated leader sender.

Stage 2 (Computation on Labels and Result): At this stage, the leader sender receives the extracted labels from all other senders. The leader sums its own label ciphertext along with others, producing a combined label-collection ciphertext $\{\text{resLab}_i\}_{i \in L}$, where L is the set containing all the labels associated with the senders' element that matched query y . The leader then homomorphically evaluates the function $f(\text{resLab}_i; \Theta)$ to obtain $\mathbf{z}$, which is transmitted back to the receiver. The receiver decrypts $\mathbf{z}$ to obtain the final value (e.g., a numeric score), which can be a probability score or classification threshold.

Challenges for the Strawman Design. The strawman protocol provides a concrete realization of label selection and analytics under FHE. However, there are several significant challenges that must be addressed to ensure both correctness and efficiency in the strawman design. First, the core label-selection operation relies on a piecewise transformation (i.e., computing VAF), which homomorphically approximates the

indicator function $1_{x=0}$ which outputs 1 if $x = 0$, and 0 otherwise. In the previous constructions [51], [96], [53], coarse-grained precision sufficed during such approximation, which served to only distinguish intersection vs. non-intersection. Our setting requires a much higher-fidelity approximation of 1s and 0s: once an ID is matched with the query, the corresponding label must be recovered with a sufficiently small error so that further computations on these labels remain accurate. This approximation challenge becomes more severe as the bit length of identifiers reaches 64 or 128 bits, significantly increasing the computational and communication overhead required to handle such large numeric domains. Next, beyond aggregating private label retrieval, our protocol evaluates a downstream function over the extracted sensitive labels. For entries not present in the intersection, the protocol must be able to exclude their label contributions altogether and signal the receiver to exclude the computation result for non-matched labels. We note that we focus on how to deal with labels resulting from non-matching items and not on ensuring the correct evaluation of the function itself. The verification of the computation of the correct function is an inherently hard problem in FHE and a separate line of research [97].

VI. OUR PROTOCOL

In this section, we present our techniques to solve the challenges associated with the strawman protocol. First, we motivate the label selection module of our protocol from existing works [51], [52], [75] and introduce a novel approximation for VAF that achieves provably higher accuracy with significantly lower computational and depth overhead (Section VI-A). Second, we propose slot-wise windowing, an efficient method for handling large set items (e.g., $\delta = 64$ or 128), which outperforms directly designing a VAF for δ -sized large domains (Section VI-B). This enables our protocol to scale to billions of identifier-label pairs with negligible false positives. Third, we present our method for label retrieval and optimized downstream computation on matched identifiers, achieving reduced memory overhead while revealing only the final result to the receiver (Section VI-C). Finally, we introduce a method to suppress labels from non-matching items, ensuring they do not affect downstream computations in Section VI-D.

A. Novel Approximation for the VAF

The VAF plays a crucial role in our protocol. In particular, it requires a VAF with indicator-level fidelity: it must evaluate to (near) one at exact equality and to negligibly small values elsewhere, so that slot-wise multiplication cleanly extracts labels and preserves accuracy for downstream computation. We present a novel VAF built from two independent components, (i) weak domain-extension polynomials (wDEPs) that compress wide input ranges without endpoint saturation and (ii) bell-shaped functions that concentrate mass at zero. We provide closed-form bounds on approximation error, multiplicative depth, and coefficient growth, enabling high-accuracy selection for 64–128-bit identifiers under CKKS.

Relation to prior VAFs: Prior VAF-based constructions for set-intersection (e.g., KTSJ24 [51]) target membership test and they approximate a tanh-derived curve via Chebyshev polynomials, then repeatedly square/scale the output and apply DEPs for large domains, reporting empirically tuned parameters. That line does not offer label extraction, encrypted real-valued analytics, or analytic guarantees needed for our pipeline. In contrast, our design is tailored to encrypted label selection and computation, with provable accuracy and complexity bounds and compatibility with the end-to-end CKKS setting.

1) Revisiting Domain Extension Polynomials: The DEP $D(x)$ shrinks the domain $[-L^n R, L^n R]$ to a smaller one $[-R, R]$ by behaving $D(x) = x$ when $x \in [-R, R]$, while $D(x) = R$ ($D(x) = -R$, resp.) for $x > R$ ($x < -R$, resp.). Then for a function f defined over $[-R, R]$, the composition $f \circ D$ becomes an extension of f to a larger domain $[-L^n R, L^n R]$ by regarding all the value outside of $[-R, R]$ as $f(R)$ or $f(-R)$ depending on the input's sign, while maintaining $(f \circ D)(x) = f(x)$ for $x \in [-R, R]$. However, when designing VAFs, we observed that it is not necessary for $D(x)$ to behave like the identity function near $[-R, R]$; in fact, the condition $D(x) = 0$ iff $x = 0$ is enough. Instead, to facilitate separating zero and nonzero inputs after applying the DEP, we focus on quantifying how far output values are from 0 for nonzero inputs. From these motivations, we propose a new definition of DEPs tailored for our purpose, called *weak DEPs* (wDEPs), as follows:

Definition 2. For $0 < R_1 < R_2$, $0 < T < R_2$, and $\epsilon_T > 0$, we define $\mathcal{D}(R_1, R_2, T, \epsilon_T)$ as a class of polynomials $p : [-R_2, R_2] \rightarrow [-R_1, R_1]$ satisfying the following properties:

- $p(x) = 0$ if and only if $x = 0$.
- $\epsilon_T < |p(x)| < R_1$, $\forall x \in [-R_2, -T) \cup (T, R_2]$.

We call $\frac{R_2}{R_1}$ a domain extension ratio. We say that the elements of $\mathcal{D}(R_1, R_2, T, \epsilon_T)$ is wDEP from $[-R_2, R_2]$ to $[-R_1, R_1]$.

The key difference between our wDEP and the original DEP [53] is that ours does not require the polynomial to behave like an identity mapping, e.g., conditions from [53] such as $|x - p(x)| < \omega|x|^3$ for some parameter $\omega > 0$ and $0 \leq p'(x) \leq 1$ for some small interval $(-r, r)$. Instead, we focus on quantifying the deviation of the function at the *tail* part of the domain, i.e., outside of $[-T, T]$.

Construction of wDEPs can be done by the same domain extension method in the original DEP [53], which sequentially maps $x \mapsto L^i D(\frac{x}{L^i})$ for $i = n-1, n-2, \dots, 0$ and some base function $D(x)$. For the extension ratio $L > 1$, we found that the following conditions are enough as a base function.

- $D(x) = 0$ if and only if $x = 0$.
- $|D(x)| \leq 1$, $\forall x \in [-L, L]$.
- $\exists x_1, x_2 \in (-L, L)$ s.t. $D(x_1) = -1$ and $D(x_2) = 1$.

We found that $D(x; k) = \frac{3\sqrt{3}}{2k\sqrt{k}}x(k - x^2)$ satisfies all the conditions. In particular, for $\max\{\sqrt{k/3}, 1\} < L < \sqrt{k}$, we can show that $D(x; k) \in \mathcal{D}(1, L, \sqrt{k/3}, D(L; k))$. The justifications behind these conditions and our choice of the function are provided in Section A and B.

Separation Factor: To quantify how much the given wDEP $f_{\text{wDEP}} \in \mathcal{D}(R_1, R_2, T, \epsilon_T)$ separates values from 0, we define $\min \left\{ |f_{\text{wDEP}}(x)| : x \in [-R_2, R_2] \cap (\mathbb{Z} - \{0\}) \right\}$ as a *separation factor*, i.e., the closest evaluation value to 0 for nonzero integer inputs. Since the image of f_{wDEP} becomes the domain of the subsequent function, and the input of wDEP is expected to be an integer, the set $\{f_{\text{wDEP}}(x) : x \in [-R_2, R_2] \cap \mathbb{Z}\}$ determines the input domain of the subsequent function defined over the range $[-R_1, R_1]$. We provide more analysis and an efficient method to compute it in Appendix A.

2) **VAFs from Bell-Shaped Functions:** For a wDEP $f_{\text{wDEP}} \in \mathcal{D}(R_1, R_2, T, \epsilon_T)$ with a separation factor ϵ_{sep} , we can ensure that the inputs of VAF from nonzero inputs never fall into $(-\epsilon_{\text{sep}}, \epsilon_{\text{sep}})$. Hence, for a function f_{BS} defined over $[-R_1, R_1]$ such that the value outside the domain $(-\epsilon_{\text{sep}}, \epsilon_{\text{sep}})$ is sufficiently small to 0, we can observe that the composition $f_{\text{BS}} \circ f_{\text{wDEP}}$ behaves like the desired VAF. We formally define such a function as a (B, ϵ) -bell-shaped function, where the parameters ϵ and B correspond to ϵ_{sep} and the maximum evaluation value outside $(-\epsilon, \epsilon)$.

Definition 3. Let $f : [-1, 1] \rightarrow [0, 1]$ be a function. For $B, \epsilon \in (0, 1)$, f is (B, ϵ) -bell-shaped if the following properties hold:

- $f(x) = f(-x)$, $f(0) = 1$, and $f(\epsilon) = B$.
- $\forall x \in [-1, \epsilon) \cup (\epsilon, 1]$, $0 \leq f(x) \leq B$, otherwise $f(x) \geq B$.

Transformation between Bell-shaped Functions: There would be several ways to design bell-shaped functions. Among them, we consider repeatedly applying the transformations between bell-shaped functions. For example, the squaring operation can be viewed as a transformation from (B, ϵ) -bell-shaped function $f(x)$ to another (B', ϵ') -bell-shaped function $g(x) = f(x)^2$ with $B' = B^2$ and $\epsilon' < \epsilon$. However, such a naïve squaring results in a vanishing of B before obtaining a sufficiently small ϵ . If $B = 0.5$ and we apply 12 squaring operations, then the final B becomes $B^{2^{12}} = 2^{-4096}$, which is too small to handle in usual parameters in CKKS.

To address the above issue, we aim to find a transformation that ensures $B' = B$ while reducing ϵ' as much as possible. To this end, we consider an affine transformation before squaring: $g(x) = (a \cdot f(x) + b)^2$ for some parameters a, b such that $a \geq 1$ and $a + b = 1$. We can observe that the range of $a \cdot f(x) + b$ in $[-1, -\epsilon) \cup (\epsilon, 1]$ becomes $[b, a \cdot B + b]$. Thus, the bound parameter B' of g becomes $\max\{b^2, (a \cdot B + b)^2\}$.

To reduce ϵ' as much as possible, we first show that ϵ is strongly related to the second derivative of f . If we consider a Taylor approximation of f at $x = 0$, we obtain $f(x) = 1 + \frac{f''(0)}{2}x^2 + O(x^4)$. This gives an approximation of the solution of $f(x) = B$ as $\sqrt{\frac{1-B}{-f''(0)}}$. Hence, increasing $|f''(0)|$ results in reducing ϵ . On the other hand, with a simple calculation, we can observe that $g''(0) = 2a \cdot f''(0)$. This implies that we need to select the largest possible a while maintaining the same B . To this end, we can select $a = 1 + \sqrt{B}$ and $b = -\sqrt{B}$; in this case $\max\{(a \cdot B + b)^2, b^2\} = b^2 = B$. Note that this enforces $\sqrt{B} < 1$ due to the definition, hence $a < 2$.

Depth-Efficient Transformation from Lazy Division: From the above approach, one caveat is that computing $(a \cdot f(x) + b)^2$ from $f(x)$ consumes two multiplicative depths in CKKS. Since $a < 2$ due to the parameter selection, squaring it two times greatly increases the second derivative. However, when a and b are rational numbers, we can mitigate this issue by applying a lazy division on their denominators. More precisely, let us denote $a = 1 + \frac{d}{q}$, $b = -\frac{d}{q}$ for some $d, q \in \mathbb{N}$ and $d < q$, $g_0(x) = f(x)$, and $g_{n-1}(x) = (a \cdot g_{n-1}(x) + b)^2$ for $n \in \mathbb{N}$. Here, instead of sequentially computing $g_2(x) = (a(a \cdot g_0(x) + b)^2 + b)^2$ from previous functions, we can apply the division by the denominator in a lazy manner to exploit the fact that the scalar multiplication by an integer can be done without any depth consumption. As we can save one depth for each procedure except for the last one, we can compute $g_n(x)$ from $g_0(x)$ with a total depth of $(n+1)$. With this technique, our transformation can increase the second derivative much faster than squaring under the same depth consumption. More precisely, we can observe that $g_n''(x) = 2^n(1 + \frac{d}{q})^n f''(0)$, while squaring $(n+1)$ times increases the second derivative by a factor 2^{n+1} . Thus, as long as $(1 + \frac{d}{q})^n \geq 2$, we can expect that this method reduces ϵ much faster than squaring.

3) **Putting Them All Together:** By composing the wDEP and bell-shaped function with appropriate parameters, we finally construct the desired VAF over the given range. We can ensure that the nonzero evaluation results are, at most, the bound parameter of the bell-shaped function, which can be precisely obtained in our approach. We summarize and visualize our result in Theorem 1 and Figure 4, respectively.

Theorem 1. Let $f_{\text{wDEP}} : [-M, M] \rightarrow [-1, 1]$ be a weak DEP with a separation factor of ϵ_{sep} and $f_{\text{BS}} : [-1, 1] \rightarrow [0, 1]$ be a $(B, \epsilon_{\text{BS}})$ -bell-shaped function. If $\epsilon_{\text{BS}} \leq \epsilon_{\text{sep}}$, then the composition $g = f_{\text{BS}} \circ f_{\text{wDEP}}$ of them has the following property: $g(0) = 1$ and $|g(x)| \leq B$ for any $x \in [-M, M] \cap (\mathbb{Z} - \{0\})$.

We will use the following components to implement the proposed VAF. First, for the wDEP, we apply the domain extension technique for the base function $D(x; k) = \frac{3\sqrt{3}}{2k\sqrt{k}}(k - x^2)$. For the bell-shaped function, we apply our transformation $f(x) \mapsto (\frac{3}{2}f(x) - \frac{1}{2})^2$ several times on the function $f(x) = (1 - \frac{3}{2}x^2)^2$, which is a $(0.25, \frac{1}{\sqrt{3}})$ bell-shaped function. We additionally square the result five successive times to ensure that the final bound parameter is reduced to 2^{-64} . We provide detailed parameters and various presets in Appendix A.

B. Supporting Long Items via Slot-wise Windowing

To achieve negligible false positives when encoding identifiers, such as when using hash-based representations, each identifier must be encoded with a sufficiently large bit-length to minimize collisions. For instance, using $\delta = 64$ or 128-bit encoding provides a robust guarantee against such collisions for a very large number of items. In previous PSI constructions based on finite-field FHE [98], [35], they encountered similar performance barriers for longer items (i.e., large bit-lengths δ). As δ grows, FHE parameters must be increased to avoid overflows and to preserve security, greatly increasing

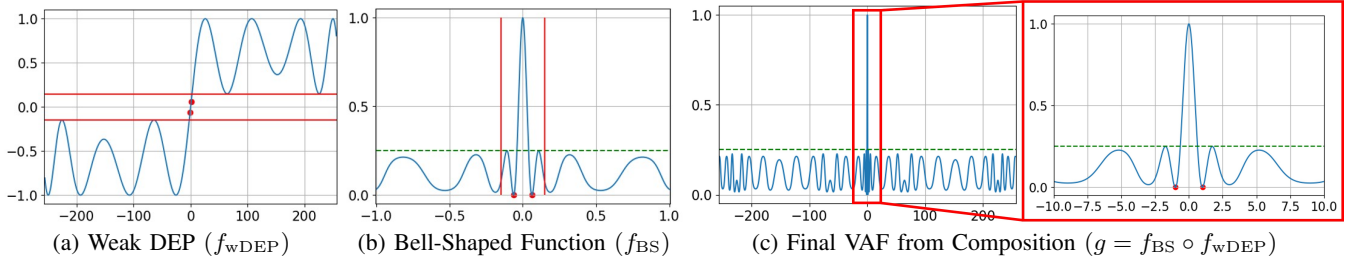


Fig. 4: Visualization of the proposed VAF construction. The red dots correspond to intermediate values for evaluating $g(1)$ and $g(-1)$, and the red solid lines and green dashed lines correspond to ϵ_T and B , respectively. We can observe that for all nonzero integers x in the domain, $g(x) \leq B$ holds.

overhead. CHLR18 [35] addressed this issue by splitting large items into multiple smaller chunks and exploiting special SIMD encodings [99] that handle larger bit length items more efficiently. Even though we use CKKS, we face a similar problem during the approximation as approximating over large numeric ranges for high δ values is very costly, demanding substantial computation and FHE depth. Our novel VAF construction faces the same problem despite several improvements in approximation. As the depth of VAF evaluations significantly impacts the concrete efficiency, both in terms of communication and computation costs, it is necessary to optimize this core and computationally intensive component.

To address this issue, we propose a windowing technique for long items called *slot-wise windowing*. This method enables us to design a VAF for large domains using those defined over much smaller domains. For a precise description, let us assume that the sender's set elements (identifiers) are of some bit length, say δ -bits. Our key strategy is to parse the items into κ parts, so that each segment is represented by at most $\xi := \lceil \frac{\delta}{\kappa} \rceil$ bits. That is, we divide each identifier x into κ components $x_1, \dots, x_\kappa$. Note that $x = 0$ if and only if $x_1 = \dots = x_\kappa = 0$. Now, let $f_{\text{VAF},\xi}$ be a VAF defined for ξ -bit inputs. Then we can rewrite the previous condition by $x = 0$ if and only if $f_{\text{VAF},\xi}(x_i) = 1$ for all $i = 1, \dots, \kappa$. This motivates us to build a function $f_{\text{VAF}}(x) := \prod_{i=1}^{\kappa} f_{\text{VAF},\xi}(x_i)$. We can immediately check that this function is indeed a VAF over the domain $[-2^\delta, 2^\delta]$. The choice of κ results in different communication-computation trade-offs in our protocol. We refer to Appendix B for more analysis and a concrete algorithm.

We can observe that the above f_{VAF} is more depth-efficient than one directly built through techniques in Section VI-A, even in smaller domains. This is because κ evaluations of $f_{\text{VAF},\xi}$ can be computed in parallel, and the final multiplication only takes $\log_2 \kappa$ depths. We note that more slots are required to represent each set item as κ grows, which reduces throughput. In Section VIII-B3, we show that with a careful choice of κ , our windowing method provides robust throughput even when dealing with smaller domains.

C. Label Retrieval and Downstream Computation

We first introduce a method to optimize the downstream computation phase after the VAF-based label-retrieval on

matched identifiers. Next, we address the key challenge from Section V by eliminating labels from non-matching items.

As detailed in Section V, our VAF enables each sender to efficiently extract labels by producing an *indicator* (1 for matched ID) that extracts the label from a target slot of label ciphertext via multiplication. The ciphertexts containing the labels after the extraction stage are then sent to one of the senders (leader). If there is a match, the label can reside in any one of the ciphertext slots; otherwise, all slots contain zeroes. To enable computations on these labels, the leader sender needs to be informed about the location of the slot that contains the extracted labels. Since the leader doesn't know which slot the label resides in, the leader can simply perform $\log(\eta)$ rotations and additions to accumulate the values in all slots of resLab_i , such that each slot of resLab_i contains the label (in case of a match) with some loss in precision which can be mitigated by an accurate VAF design. The leader can then operate on n -label ciphertexts, resLab_i , to compute any function homomorphically.

We observed that if the leader is able to aggregate multiple label ciphertexts into a single ciphertext, it can help reduce the memory overhead during computation. Aggregation enables the leader sender to operate on η slots of a single ciphertext instead of operating on n FHE ciphertexts, which can be costly in terms of memory. However, aggregating results label ciphertexts by the leader is not trivial: naïve additive aggregation fails due to unknown and overlapping slot positions. We resolve this with a technique we call *slot isolation*.

Optimizing Memory Overhead via Slot Isolation: First, each sender locally performs the $\log(\eta)$ slot rotations and additions to assemble resLab_i before transmission, parallelizing the costly FHE operations and removing the leader as a computation bottleneck. Next, each sender multiplies resLab_i by oneHot_i plaintext vector, which has all zeroes except a single 1 in slot i , yielding a ciphertext whose label for i -th sender is isolated in slot i . If there are multiple types of labels (say $|\mathcal{L}|$), then oneHot_i contains exactly $|\mathcal{L}|$ slots containing 1. The leader and other senders are informed about the number of multiple types of labels held by each sender at the setup phase. The leader homomorphically aggregates its own extracted label, along with all other resLab_i , to obtain $z = \sum_{i=1}^{n-1} \text{resLab}_i$, which contains the label for the i -th sender at the i -th or multiple $|\mathcal{L}|$ slots. Only one ciphertext

z is then used during the function computation, instead of n such ciphertexts, reducing the memory overhead from n FHE ciphertexts to one. This optimization requires $n < \eta$ when each sender contributes a single label type; if senders contribute multiple types, the number of distinct label types must instead satisfy $|\mathcal{L}| < \eta$. In practice, both n and $|\mathcal{L}|$ are typically well below η in most practical multi-party scenarios.

D. Dealing with Labels from Non-Match IDs

In our construction, the leader sender may receive *empty* ciphertexts (i.e., encryptions of the zero vector) when no intersection occurs. Since label values are unrestricted in our construction, a true label value of “0” may be indistinguishable from the near-zero output of a non-intersection during the function computation phase. On the other hand, when the distribution of labels is far from 0, providing “0” value to the function may result in an invalid computation outcome. Thus, appropriate mitigation is required to ensure correctness in the presence of empty ciphertexts.

We present two methods to preclude this issue. First, each sender designates one slot in common and embeds the intersection result into that slot, i.e., the summation of VAF results across all the data. Then, after aggregation, the resulting ciphertext contains the number of matches across all senders. The leader sender may apply another VAF to this number. With this idea, the receiver can decide whether the retrieved evaluation value was derived from actual matches or not. In addition, we also employ *imputation*, a common method in statistical analysis for dealing with missing variables [100]. In this technique, the missing variables are replaced by the statistics of the data, such as mean or median. In our scenario, these values can be pre-computed and delegated by the data owner. We observe that by viewing the evaluation result of the VAF as an *indicator*, each sender can decide whether to use the original label or the statistic value during label extraction. Hence, function evaluation becomes robust to missing values across senders, yielding reliable results.

VII. DISCUSSION

a) *Eliminating TTP via Threshold FHE*: We remove the TTP assumption by instantiating our full protocol using an α -out-of- n CKKS thresFHE with noise flooding to achieve IND-CPA^D security that tolerates up to $\alpha - 1$ colluding parties for $\alpha < n/2$. ThresFHE limits the full-exposure of the decryption key to a single party and removes the single point of failure by preventing any sub-threshold coalition from decrypting or attributing matches. It also strengthens data-owner trust in delegated storage as no single party, including the receiver, can recover plaintext ID-label pairs, and decryption is possible only to an authorized threshold coalition.

b) *Duplicate Identifiers and Label Types*: Our model assumes that duplicate IDs may exist across different senders’ sets but not within a single sender’s set. Data owners can simply preprocess the ID-label pairs before uploading them to the senders to achieve this, and this practice matches standard data-cleaning pipelines that remove duplicated data

and noise before any privacy-preserving linkage [101]. Each sender packs its retrieved label into one CKKS ciphertext, and the leader evaluates f once per ciphertext, yielding $O(n)$ cost regardless of duplicate ownership. We permit each sender to include multiple types of labels in their dataset, but all senders share the same overall label schema (i.e., they hold labels of the same types). Our framework supports variable label cardinality per ID across different senders.

c) *Provenance Security*: Many works have discussed scenarios where the origin of an intersecting item must remain undisclosed, sometimes referring to it informally as “source anonymity” or “provenance privacy” [102], [51], [103], [104], [105]. We can ensure provenance security in our protocol by permuting the slots in $resLab_i$ after the label retrieval, which hides the mapping between the senders and the slots. However, this makes the protocol inherently restricted to symmetric functions (summation, cardinality, etc.). Prior works [40] operate under this setting for computing over the intersection, thereby preserving item-level and provenance security. However, supporting asymmetric or general-purpose functions can offer significantly richer functionalities [18]. While asymmetric function computation may leak provenance, such leakage is limited to the leader. The receiver learns nothing unless it colludes with the leader. In practice, the leader is incentivized, both reputationally and economically, to preserve the provenance security of senders and ensure correct output. We provide a formal definition and proof for *provenance security*, which requires that adversarial parties cannot link items in the intersection back to the dataset owner(s), and the leakage is only limited to the party who is responsible for function computation. We provide the formal definition for such security in Definition 4 in Appendix E.

Our complete ELSA protocol is detailed in Figure 5 after incorporating the aforementioned optimizations and mitigations. We prove the protocol secure in the semi-honest model against up to $(\alpha - 1)$ colluding senders and the receiver, with a simulation-based proof in Appendix E.

VIII. EVALUATION

A. Experiment Setup

We implement our protocol in C++17, using OpenFHE v1.2.3 [106]. The end-to-end protocol is instantiated with thresFHE CKKS and incorporates slot-wise windowing, slot isolation, and imputation for non-match IDs. The anonymized source code is available at <https://anonymous.4open.science/r/ELSA-4242>. Our tests were run on a single Intel Xeon(R) Gold 5412U server (512GB RAM, Ubuntu 22.04). Each experiment was repeated 10 times, and we report the average run-time. We used default thresFHE CKKS parameters in OpenFHE to maintain 128-bit classical security [107]. We assumed $\alpha = n/2$ for thresFHE and FHE ring dimension, $N = 2^{17}$. Unless otherwise specified, we treat each dataset item as a hashed value of length $\delta = 64$. We re-implemented all of the compared works, [36], [37], and [51], in OpenFHE under identical FHE parameter settings for a fair comparison. Each sender’s computations are independent of each other in

Parameters: Each sender $i \in [1, n-1]$ receives encrypted inputs $(c_{x_i}, c_{\ell_{b_i}})$, where X_i and ℓ_{b_i} are the input set and associated labels, and y is the receiver's input. λ is the computational security param., δ is the bit-length of set elements; both are public. The slot-wise windowing parameter $\kappa \in \mathbb{N}$ and two VAFs, $f_{\text{VAF}}^{\text{int}}$ and $f_{\text{VAF}}^{\text{agg}}$, defined over domains $[-2^{\delta/\kappa}, 2^{\delta/\kappa}]$ and $[-n, n]$ and the bound error ν , respectively, are known publicly. Senders possess $f(\{\ell_{b_i}\}_{i \in L}; \Theta)$, where Θ is some private parameter and L contains the labels associated with y .

1. **[Parameters]**

- Threshold FHE:** Parties, including the receiver and senders, agree on parameters $(N, q, \delta, D, \alpha)$ for the *ThresFHE* CKKS scheme.
- Key Distribution and Setup:** A subset of parties ($\alpha - 1$ senders and receiver) run a secure multiparty computation protocol *ThresFHE.KeyGen* that provides each partial key shareholder $i \in [1, \alpha)$ a share of the secret key sk as one of $\{sk_1, sk_2, \dots, sk_\alpha\}$ and broadcasts the common public key pk and evaluation key evk to all other parties. Senders jointly agree on permutation, obtaining a ciphertext $perm_i$ that contains 1 in ρ_i -th slot and 0 elsewhere. Alternatively, a trusted setup can compute the common encryption, partial, evaluation keys, and $perm_i$ for each sender. All the parties, including data owners, have access to the public encryption key pk and evaluation key evk after this step.

2. **[Encryption]**

- Each data owner $i \in \{1, \dots, n-1\}$ organizes its input vectors $x \in X_i$ and corresponding labels $\ell \in \ell_{b_i}$ into $\mathbb{R}^m$. These vectors are then divided into κ parts, which are then sequentially packed into κ plaintexts of size $N/2$, producing $\kappa \times \frac{2m}{N}$ plaintexts, which are encrypted using *ThresFHE.Enc*. Consequently, κ ciphertext slots holds an individual x and ℓ value. In addition, it computes the statistics $\ell_{b_{\text{stat}}}$. Every data owner encrypts its sets and the statistics value independently and forwards the resulting ciphertexts to the appropriate senders $i \in \{1, \dots, n-1\}$. After receiving the ciphertext pair(s) $(c_{x_i}, c_{\ell_{b_i}})$ and the statistics ciphertext $c_{\ell_{b_{\text{stat}}}}$, sender i may hold multiple ciphertexts if $|X_i| > \eta$.
- Encrypt replicas of y :** Receiver constructs a vector of length η , with each element being $y \in \mathbb{R}$. Then, it divides the vector into κ segments, sequentially packs the first η elements into a CKKS plaintext, and encrypts it using *ThresFHE.Enc*. As a result, each consecutive κ slots of the resulting ciphertext c_y contains a single y value.

3. **[Compute Intersection]**

For each pair $(c_{x_i}, c_{\ell_{b_i}})$, and the ciphertext c_y , sender i uses *ThresFHE.Eval* with evk to execute the following steps:

- Computes $diff_i = c_y - c_{x_i}$. Next, it applies a piecewise function $select_i = f_{\text{VAF}}^{\text{int}}(diff_i)$ using evk and obtains $select_i$. If possessing multiple set ciphertexts, the sender computes multiple $select_i$ (s) in parallel.
- Applies rotation-and-multiplication technique to multiply consecutive κ slots in each $select_i$.
- Homomorphically multiplies a masking vector that comprised by $\frac{\eta}{\kappa}$ concatenations of $(1, 0, \dots, 0) \in \mathbb{R}^\kappa$ in each $select_i$.

4. **[Label(s) Retrieval and (Optional) Permutation]**

For each pair $(c_{x_i}, c_{\ell_{b_i}})$ and $select_i$, sender i homomorphically executes the following:

- Computes $resLab_i = select_i \times c_{\ell_{b_i}}$ where $c_{\ell_{b_i}}$ corresponds to c_{x_i} . If possessing multiple $resLab_i$, the sender sums them to a single $resLab_i$.
- Rotates and adds $resLab_i$ and $select_i$ with $\log(\eta)$ operations to replicate intersection label and indicator for intersection in all slots. The result from the latter is set to $flag_i$. Then it computes $resLab_i + (\vec{1} - flag_i) \times c_{\ell_{b_{\text{stat}}}}$ where $\vec{1} \in \mathbb{R}^{N/2}$ is a vector filled with 1.
- If f is a symmetric function sender computes $resLab_i \times perm_i$, where $perm_i$ permutes, and isolates label in a single slot of $resLab_i$. Otherwise, sender ignores this step.

Finally, each sender transmits $resLab_i$ and $flag_i$ to leader.

5. **[Computation on Retrieved Label(s)]**

The leader sender obtains $\{resLab_i, flag_i\}_{i \in [n-1]}$ from all the senders and computes the following:

- Computes $resLab = \sum_{i=0}^{n-1} resLab_i$. Then, using evk and private parameter Θ , the leader computes $\mathbf{z} = f(resLab_i; \Theta)$, where f operates labels located at the slots of $resLab$.
- Computes $flag = \sum_{i=0}^{n-1} flag_i$ and homomorphically evaluates $flag = f_{\text{VAF}}^{\text{agg}}(flag)$.

The leader sender returns $\mathbf{z}$ and $flag$ to the receiver and $\alpha - 1$ senders for partial decryption.

6. **[Partial and Final Decryption]**

- Partial decryption & smudging noise :** Upon receiving $\mathbf{z}$ and $flag$, each partial key-share holder sender $i \in [1, \alpha)$ uses their secret key share sk_i to partially decrypt them separately using *ThresFHE.PartialDec*, and introduces a smudging noise e_{smg} to the partial decryptions $part_i$ and $part_{flag_i}$. Each sender i computes $part_i$ and $part_{flag_i}$ and sends their decryption share to the receiver.
- Final decryption:** The receiver uses their partial decryption key sk_i and $\mathbf{z}$ and $flag$ to execute *ThresFHE.PartialDec* and obtains their partial decryption share $part_i$ and $part_{flag_i}$. Optionally, the receiver can use a *ThresFHE.PartialDec* algorithm that does not introduce a smudging noise if they do not wish to share the result with any of the senders. Receiver combines their share with partial decryptions received from the $\alpha - 1$ senders to compute $\{part_i : sk_i\}_{i \in [0, \alpha)}$ and $\{part_{flag_i} : sk_i\}_{i \in [0, \alpha)}$ using *ThresFHE.Combine* and pk and obtains the resulting vector of length η .
- Interpretation of result:** If the decryption of $flag$ is 0, then the receiver rejects the result from the decryption of $\mathbf{z}$. Otherwise, the receiver confirms the result of $f(\{\ell_{b_i}\}_{i \in L}; \Theta)$.

Fig. 5: Full ELSA protocol without a Trusted Third Party

the multi-sender setting in our protocol. Once they receive the query ciphertext from the receiver, we assume each sender processes its own encrypted data in parallel. For noise smudging, we assume that senders perform a one-time, offline static noise estimation using publicly available input bounds based on real data and the evaluated VAF. This process depends solely on query computation and incurs negligible latency. In our LAN configuration, we assume a 20 Gbps and 10 Gbps bandwidth

with a 0.2 ms RTT, while the WAN setup assumes 1 Gbps and 500 Mbps bandwidths with an 80 ms RTT latency.

Since there is no direct comparison for the same end-to-end functionality as ours, we compare only the selection stage against ELSA to state-of-the-art baselines. We evaluate the performance in terms of computational and communication costs and scalability in terms of the number of senders and set sizes for two variants of our protocol, (1) a selection stage-

only benchmark and comparison against state-of-the-art prior works in Section VIII-B and Section VIII-B3, (2) a labeled version (i.e., ELSA with downstream computations) evaluation on three fraud-oriented datasets provided by Fraud Dataset Benchmark (FDB) ¹ in Section VIII-C. These datasets were selected to evaluate the end-to-end performance of ELSA under high label and entry counts. The IEEE-CIS Fraud Detection dataset (IEEECIS) comprises around 590K card-not-present transactions with 25 labels. The Sparkov (Simulated) Credit Transactions dataset (CCTFD) includes 1.2 million entries with 24 labels, and the Vehicle Loan Default dataset (VLDP) consists of about 233K records with 44 labels. We provide additional domain-specific and preprocessing details of these datasets in Appendix D. For the downstream computations, we employed logistic regression, a widely used model in applications such as fraud detection and disease prediction [108]. We assume senders have access to a pre-trained logistic regression model $\sigma(\mathbf{w}^\top \mathbf{y} + b)$ which is parameterized by some private parameter Θ (e.g., a weight vector $\mathbf{w} \in \mathbb{R}^d$ and bias $b \in \mathbb{R}$) in one instantiation.

B. Benchmarking the Selection Stage (Query Matching)

In this section, we benchmark the performance metrics of the selection stage in ELSA, under varying numbers of senders and encrypted sender set sizes. We first discuss the preference criteria of the slot-wise windowing parameter κ . We then outline end-to-end latency in terms of both computational and communication, demonstrating our approach’s scalability.

1) *Evaluating Optimal κ for Supporting Large Items:* Choosing optimal κ in the slot-wise windowing method for a particular sender set size is essential in our method. In Table II, we evaluate the impact of varying κ on communication, computation, and precision when approximating the VAF over a 2^{16} -sized sender set with 20-bit elements under a single-thread setting. The optimal value of κ in this setting can be determined to be either 5 or 10 (highlighted). $\kappa = 5$ reduces the sender storage but $\kappa = 10$ reduces the communication cost. Both achieve the lowest latencies while preserving high precision. For the evaluation of the datasets with a larger amount of records and δ value (as in FDB), we require different κ values and refer to Appendix B for more details (e.g., when dealing with $\delta = 64$ or 128). As a baseline, we compare our optimization method against the state-of-the-art work that conducts CKKS-based approximation for membership testing.

2) *Scalability in terms of Number of Senders and Set Sizes:* The total latency (including communication cost) of our approach for an increasing number of senders under various network bandwidths is shown in Figure 6 (a). We can observe that our core selection stage easily scales for a high number of senders (up to 4096). For the applications discussed in Section I, the institutions are often equipped with high bandwidth LAN connections, and our total latency stays below 400 sec under such connections. We show the total end-to-end latency for various total sender set sizes in Figure 6 (b).

TABLE II: Comparing methods to approximate VAF over a single sender with 2^{16} . $\kappa = 1$ denotes slot-wise windowing is disabled. Ctxt denotes FHE ciphertext.

Approx. Method	Item Length (δ)	Sender Ctxt No.	Receiver Ctxt (MB)	FHE Depth	CKKS Precision (bits)	Time (s)
KTSJ24	20	1	113.3	52	30.0	151.6
$\kappa = 1$	20	1	86.0	39	25.1	28.87
$\kappa = 2$	20	2	54.0	24	36.0	11.37
$\kappa = 4$	20	4	44.0	19	42.6	8.20
$\kappa = 5$	20	5	37.0	16	39.6	5.60
$\kappa = 10$	20	10	33.0	14	42.1	5.48

Our method can easily support up to 2^{30} sender set size, and the overall latency remains below 150 sec under LAN settings.

3) *Comparison with State-of-the-art:* In this section, we compare our core functionality (query matching on encrypted records) against various state-of-the-art works. We evaluate the end-to-end latency (computation and communication) and show how it scales with the increasing number of senders and set sizes under a 10 Gbps LAN setting. Considering the existing works that we discussed in Section II, [54], [55], and [56] are non-FHE-based works that support only a limited number of senders. We primarily compare our method against three state-of-the-art FHE-based works, Cong *et al.* [36], PEPSI [37], and KTSJ24 [51]. Note that Cong *et al.* and PEPSI operate on plaintext sender sets in their protocols to enable various optimizations. Hence, we keep the same plaintext setting for them. Therefore, KTSJ24 and ours are disadvantaged in this comparison since they are run on encrypted sender sets. We use $\Delta = 59$ for CKKS-based methods (ours and KTSJ24). We set $\delta = 64$ for Cong *et al.*, PEPSI, and ours. For KTSJ24, we used $\delta = 25$ (the highest δ supported by their parameter presets), although it favors them. Ours, Cong *et al.*, and PEPSI achieve a negligible false positive rate (2^{-30} to 2^{-40}) from hash collisions when using 64-bit hashed items across all the sender set settings, while KTSJ24 exhibits non-negligible hash collisions. We note that ours can be easily extended to a higher δ , which will incur even smaller false positive rates ($< 2^{-100}$ when $\delta = 128$). The end-to-end latency results are shown in Figure 6 (c) and Figure 7, where the maximum number of senders and set size is capped at 5000 and 2^{30} , respectively, due to memory constraints.

Our method achieves significantly lower latency ($3.5\times - 6.8\times$) compared to Cong *et al.* and KTS24 in Figure 6 (c). Compared to PEPSI, our protocol has higher latency when the number of senders is less than 4096, but we operate on fully encrypted datasets. However, ours gets faster (projected dotted lines in Figure 6 (c)) as we cross 4096 senders. In terms of varying sender set sizes shown in Figure 7, ours is faster ($1.4\times - 5.4\times$) than other methods. PEPSI starts to gain speed up over ours for larger set sizes since they operate over plaintext sender sets and cannot support downstream computations. In summary, our protocol matches or outperforms state-of-the-art works that operate on plaintext sets while additionally enabling secure label retrieval and downstream analytics.

C. Experiments on Real-World Datasets

We benchmark the ELSA protocol with downstream logistic regression on three FDB datasets. Since ELSA supports arbi-

¹<https://github.com/amazon-science/fraud-dataset-benchmark>

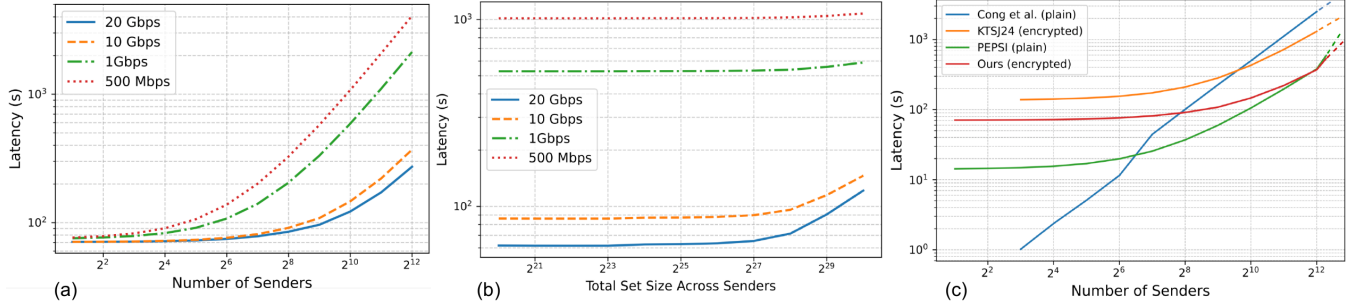


Fig. 6: (a) Latency of ELSA’s selection stage for different number of senders (2^{20} items per sender), (b) Latency of ELSA’s selection stage for different total sender set size across senders (1024 senders); $\delta = 64$ and $\kappa = 8$ in both (a) and (b). (c) Comparison of ELSA’s selection stage with prior works. Number of senders is set to 1024; $\delta = 64$ for all except KTSJ24, $\kappa = 8$ for ours. Figures are drawn on a log scale.

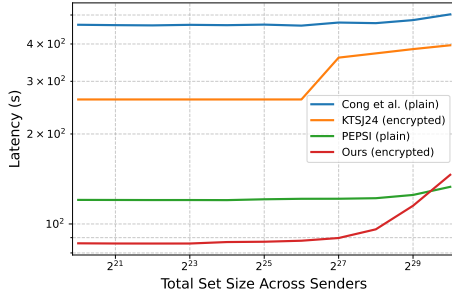


Fig. 7: Comparison of ELSA’s selection stage with other works. The number of senders is set to 1024. $\delta = 64$ for all except KTSJ24. $\kappa = 8$ for ours.

TABLE III: ELSA latency results for FDB datasets (per query). Best computation times are in bold. T denotes the number of threads. Sender count and storage refer to the number of senders and storage needed for each sender, respectively. $\delta = 64$ and $\kappa = 8$.

Dataset	Sender		Comm. Time (s)				Online Latency (s)				
	Count	Storage (MB)	20 Gbps	10 Gbps	1 Gbps	500 Mbps	T=1	T=2	T=4	T=8	T=16
VLDP	176	1291.8	9.9	19.8	197.5	395.0	168.1	150.8	92.2	62.6	63.2
CCTFD	960	738.2	53.9	107.9	1078.3	2157.4	128.6	104.1	66.9	63.5	69.8
IEEEECIS	250	1291.9	14.0	28.1	280.7	561.4	163.0	147.9	88.3	58.9	62.9

rary computations on encrypted real-valued labels and scales to a very large number of senders, selecting a protocol as a case for comparison is a complex task. We thus microbenchmark ELSA for latency and communication overhead and discuss some of the limitations of prior works for comparison in our setting. Existing FHE-based works that support downstream computations after query matching, such as [35],[36], and [41] only support computation over the intersection. Other works such as [62], [45], [70], [48], and [80] are based on primitives other than FHE like SMPC or garbled circuits and

TABLE IV: Runtime percentage of FHE operations for a single query membership and computation in ELSA. $\kappa = 8$ and T=8.

FHE Operation	Party	Runtime %			Req. Depth
		VLDP	CCTFD	IEEEECIS	
Query Generation	Receiver	0.75	0.73	0.77	-
VAF w/ Windowing	Sender(s)	56.0	57.93	57.58	23
Label Retrieval	Sender(s)	12.38	6.82	12.78	2
Logistic Reg. / Flag	Leader	30.51	34.17	28.49	13/17
Decryption	Receiver	0.36	0.35	0.38	-

only support limited symmetric functions linear in set size, do not support floating point inputs, or are very costly in terms of communication cost for general computation functions. KTSJ24 does not support label retrieval, and PEPSI and Cong *et al.* introduce prohibitively high amounts of overhead for computation on encrypted floating-point labels due to finite-field FHE, which is not suitable for floating-point labels present in the FDB datasets.

Table III shows our protocol’s online latency under different numbers of threads, communication cost, and storage needs under varying counts of senders. We also display the communication latency for $\delta = 64$ with $\kappa = 8$ under various bandwidths. We vary the number of senders per dataset based on its label distribution and size to ensure balanced partitioning across senders. Total latency for all three datasets remains under 65 sec for $\delta = 64$. Importantly, increasing δ to 128 adds only a modest amount of additional costs for our protocol (with $\kappa = 16$). The online communication remains the same for 128-bit items, and latency is increased by about 15%, 41%, and 11%, respectively, for three FDB datasets (concrete details given in Appendix D). Compared to the plaintext computation, we achieve the same prediction results of up to 24 bits in CKKS while computing the logistic regression on the labels. We report the runtime percent of each step of ELSA for 3 datasets in Table IV. The selection stage (VAF evaluation) dominates runtime, accounting for over 50% of end-to-end latency. We note that the number of senders is kept consistent in both Table III and Table IV across different datasets.

IX. CONCLUSION

In this work, we present ELSA, an end-to-end protocol for encrypted label selection and analytics on distributed datasets. ELSA combines CKKS-based thresFHE with high-accuracy VAF approximations to support large identifier domains and real-valued labels, enabling encrypted label extraction and efficient downstream computations. We evaluate our method on real-world fraud datasets and demonstrate that our protocol is significantly better than prior work, achieving up to $6.8\times$ speed-up, while maintaining scalability and high precision. In future work, we aim to explore multi-query workloads, richer predicates, and additional system-level optimizations.

REFERENCES

- [1] M. T. McCarthy, “Usa patriot act,” 2002.
- [2] A. Alexandru and K. Rohloff, “Privacy-preserving data sharing across financial institutions,” Presented at NIST Workshop on Privacy-Enhancing Cryptography, 2024, accessed: 2025-04-02. [Online]. Available: <https://csrc.nist.gov/csrc/media/presentations/2024/wpcc2024-2a6/images-media/wpcc2024-2a6-slides-andreea-kurt--PPDS-financial.pdf>
- [3] “Anti-money laundering (aml) solution for banks,” <https://dualitytech.com/use-cases/financial-services-industry/anti-money-laundering-aml/>, Duality Technologies, accessed: 2025-05-23. [Online]. Available: <https://dualitytech.com/use-cases/financial-services-industry/anti-money-laundering-aml/>
- [4] Association of Certified Fraud Examiners, “Report to the nations: 2022 global study on occupational fraud and abuse,” 2022, median detection times and investigation durations for fraud cases.
- [5] Federal Financial Institutions Examination Council, “Bank secrecy act/anti-money laundering examination manual,” 2021, available at: <https://bsaaml.ffiec.gov/manual>.
- [6] “Bank secrecy act of 1970 (currency and foreign transactions reporting act),” Public Law 91-508, 84 Stat. 1114, 1970, codified at 31 U.S.C. §§5311–5332.
- [7] “Regulation (eu) 2016/679 of the european parliament and of the council (general data protection regulation),” Official Journal of the European Union L119/1, Apr. 2016, available at: <https://eur-lex.europa.eu/eli/reg/2016/679/oj>.
- [8] “Standards for privacy of individually identifiable health information (hipaa privacy rule),” 45 CFR Parts 160 and 164, 2002, u.S. Dept. of Health and Human Services.
- [9] Office of the National Coordinator for Health IT, “21st century cures act: Interoperability, information blocking, and the one health it certification program,” Federal Register 85 FR 25642, 2020.
- [10] J. R. Vest and L. D. Gamm, “Health information exchange: Persistent challenges and new strategies,” *Journal of the American Medical Informatics Association*, vol. 17, no. 3, pp. 288–294, 2010.
- [11] “California consumer privacy act of 2018,” Cal. Civ. Code §§1798.100–1798.199, 2018, available at: <https://oag.ca.gov/privacy/ccpa>.
- [12] National Association of REALTORS®, “Data privacy & security,” <http://www.nar.realtor/data-privacy-security>, 2017, accessed 24 Jul 2025.
- [13] E. Dauterman, M. Rathee, R. A. Popa, and I. Stoica, “Waldo: A private time-series database from function secret sharing,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 2450–2468.
- [14] N. Wang, W. Zhou, J. Wang, Y. Guo, J. Fu, and J. Liu, “Secure and efficient similarity retrieval in cloud computing based on homomorphic encryption,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 2454–2469, 2024.
- [15] C. Fu, X. Zhang, S. Ji, J. Chen, J. Wu, S. Guo, J. Zhou, A. X. Liu, and T. Wang, “Label inference attacks against vertical federated learning,” in *31st USENIX security symposium (USENIX Security 22)*, 2022, pp. 1397–1414.
- [16] L. Han, W. Luo, R. Lu, Y. Zheng, A. Yang, J. Lai, Y. Cheng, and Y. Zhang, “Privacy-preserving travel recommendation based on stay points over outsourced spatio-temporal data,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [17] J. Liagouris, V. Kalavri, M. Faisal, and M. Varia, “{SECRECY}: Secure collaborative analytics in untrusted clouds,” in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 1031–1056.
- [18] M. Dong, Y. Chen, C. Zhang, Y. Bai, and Y. Cao, “Multi-party private set operations from predicative zero-sharing,” *Cryptology ePrint Archive*, 2025.
- [19] Statista, “Number of fdic-insured banks in the u.s. in 2023: 4,470,” 2023, accessed: 2025-01-17. [Online]. Available: <https://www.statista.com/statistics/184536/number-of-fdic-insured-us-commercial-bank-institutions/>
- [20] E. Szerdocz, “How banks can unite to improve anti-money laundering detection,” [Online]. Available: <https://www.partisia.com/docs/collaborative-aml-how-banks-can-unite-to-improve-anti-money-laundering-detection/>
- [21] M. B. van Egmond, V. Dunning, S. van den Berg, T. Rooijackers, A. Sangers, T. Poppe, and J. Veldsink, “Privacy-preserving anti-money laundering using secure multi-party computation,” in *International Conference on Financial Cryptography and Data Security*. Springer, 2024, pp. 331–349.
- [22] H. Zhang, J. Hong, F. Dong, S. Drew, L. Xue, and J. Zhou, “A privacy-preserving hybrid federated learning framework for financial crime detection,” *arXiv preprint arXiv:2302.03654*, 2023.
- [23] L. Hernandez Aros, L. X. Bustamante Molano, F. Gutierrez-Portela, J. J. Moreno Hernandez, and M. S. Rodríguez Barrero, “Financial fraud detection through the application of machine learning techniques: a literature review,” *Humanities and Social Sciences Communications*, vol. 11, no. 1, pp. 1–22, 2024.
- [24] C. News, “Banks are willing to cooperate for money laundering investigations,” 2025, accessed: 2025-01-17. [Online]. Available: <https://www.cbsnews.com/news/tax-evasion-billions-offshore-fatcat-tax-reporting-loophole-senate-finance-committee-robert-brockman/>
- [25] M. Alkhalili, M. H. Qutqut, and F. Almasalha, “Investigation of applying machine learning for watch-list filtering in anti-money laundering,” *IEEE Access*, vol. 9, pp. 18 481–18 496, 2021.
- [26] Department of Homeland Security, “Pets4hse use cases,” <https://pets4hse.org/PETS4HSEUseCases.pdf>, 2022, accessed: April 14, 2025.
- [27] LexisNexis Risk Solutions, “Watchlist screening - sanctions check,” <https://risk.lexisnexis.com/financial-services/financial-crime-compliance/watchlist-screening>, 2025, accessed: April 14, 2025.
- [28] Oracle and D. Technologies, “Privacy-protected aml queries: Collaborative aml investigations across banks with encrypted queries,” 2025, accessed: 2025-03-07. [Online]. Available: <https://www.oracle.com/oc/dc/assets/CONT4168C8AB242C413693AD0413917DCE7E/native/oracle-duality-data-sheet.pdf>
- [29] Duality Technologies, “Duality Query Engine: Zero-Footprint, Privacy-Enhanced Query Platform,” <https://dualitytech.com/platform/duality-query/>, 2024, web page, accessed 23 Jul 2025.
- [30] P. K. Doppalapudi, P. Kumar, A. Murphy, C. Rougeaux, R. Stearns, S. Werner, and S. Zhang, “The fight against money laundering: Machine learning is a game changer,” McKinsey & Company, October 7, 2022, 2022, <https://www.mckinsey.com/capabilities/risk-and-resilience/our-insights/the-fight-against-money-laundering-machine-learning-is-a-game-changer>.
- [31] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A {Fault-Tolerant} abstraction for {In-Memory} cluster computing,” in *9th USENIX symposium on networked systems design and implementation (NSDI 12)*, 2012, pp. 15–28.
- [32] M. Armbrust, A. Ghodsi, R. Xin, M. Zaharia et al., “Lakehouse: a new generation of open data platforms that unify data warehousing and advanced analytics,” in *Proceedings of CIDR*, vol. 8, 2021, p. 28.
- [33] Federal Financial Institutions Examination Council (FFIEC), “Bsa/aml risk assessment: Overview,” <https://bsaaml.ffiec.gov/manual/BSAAMLRiskAssessment/01>, 2021, accessed: 2025-07-13.
- [34] Alessa, “Anti-money laundering risk scoring factors,” <https://alessa.com/blog/anti-money-laundering-risk-scoring-factors/>, 2024, accessed: 2025-07-13.
- [35] H. Chen, Z. Huang, K. Laine, and P. Rindal, “Labeled psi from fully homomorphic encryption with malicious security,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 1223–1237.
- [36] K. Cong, R. C. Moreno, M. B. da Gama, W. Dai, I. Iliashenko, K. Laine, and M. Rosenberg, “Labeled psi from homomorphic encryption with reduced computation and communication,” in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1135–1150.
- [37] R. A. Mahdavi, N. Lukas, F. Ebrahimiaghazani, T. Humphries, B. Kacsmar, J. Premkumar, X. Li, S. Oya, E. Amjadian, and F. Kerschbaum, “{PEPSI}: Practically efficient private set intersection in the unbalanced setting,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 6453–6470.
- [38] A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo, “Batch {PIR} and labeled {PSI} with oblivious ciphertext compression,” in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024, pp. 5949–5966.
- [39] Y. Huang, D. Evans, and J. Katz, “Private set intersection: Are garbled circuits better than custom protocols?” in *NDSS*, 2012.
- [40] B. Pinkas, T. Schneider, C. Weinert, and U. Wieder, “Efficient circuit-based psi via cuckoo hashing,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2018, pp. 125–157.

- [41] Y. Son and J. Jeong, "Psi with computation or circuit-psi for unbalanced sets from homomorphic encryption," in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, 2023, pp. 342–356.
- [42] N. Chandran, D. Gupta, and A. Shah, "Circuit-psi with linear complexity via relaxed batch oprf," *Proceedings on Privacy Enhancing Technologies*, 2022.
- [43] P. Miao, S. Patel, M. Raykova, K. Seth, and M. Yung, "Two-sided malicious security for private intersection-sum with cardinality," in *Annual International Cryptology Conference*. Springer, 2020, pp. 3–33.
- [44] G. Garimella, P. Mohassel, M. Rosulek, S. Sadeghian, and J. Singh, "Private set operations from oblivious switching," in *IACR international conference on public-key cryptography*. Springer, 2021, pp. 591–617.
- [45] J. Gao, N. Trieu, and A. Yanai, "Multiparty private set intersection cardinality and its applications," *Proceedings on Privacy Enhancing Technologies*, 2024.
- [46] P. Rindal and P. Schoppmann, "Vole-psi: fast oprf and circuit-psi from vector-ole," in *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 2021, pp. 901–930.
- [47] P. Buddhavarapu, A. Knox, P. Mohassel, S. Sengupta, E. Taubeneck, and V. Vlaskin, "Private matching for compute," *Cryptology ePrint Archive*, 2020.
- [48] D. Mouris, D. Masny, N. Trieu, S. Sengupta, P. Buddhavarapu, and B. Case, "Delegated private matching for compute," *Proceedings on Privacy Enhancing Technologies*, 2024.
- [49] T. Lepoint, S. Patel, M. Raykova, K. Seth, and N. Trieu, "Private join and compute from pir with default," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2021, pp. 605–634.
- [50] M. Ion, B. Kreuter, A. E. Nergiz, S. Patel, S. Saxena, K. Seth, M. Raykova, D. Shanahan, and M. Yung, "On deploying secure computing: Private intersection-sum-with-cardinality," in *2020 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2020, pp. 370–389.
- [51] N. Koirala, J. Takeshita, J. Stevens, and T. Jung, "Summation-based private segmented membership test from threshold-fully homomorphic encryption," *Proceedings on Privacy Enhancing Technologies*, 2024.
- [52] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*. Springer, 2017, pp. 409–437.
- [53] J. H. Cheon, W. Kim, and J. H. Park, "Efficient homomorphic evaluation on large intervals," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2553–2568, 2022.
- [54] N. Chandran, N. Dasgupta, D. Gupta, S. L. B. Obbattu, S. Sekar, and A. Shah, "Efficient linear multiparty psi and extensions to circuit/quorum psi," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1182–1204.
- [55] O. Nevo, N. Trieu, and A. Yanai, "Simple, fast malicious multiparty private set intersection," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 1151–1165.
- [56] A. Vadapalli, R. Henry, and I. Goldberg, "Duoram: A bandwidth-efficient distributed oram for 2-and 3-party computation," in *32nd USENIX Security Symposium*, 2023.
- [57] J. Vos, S. Pentyala, S. Golob, R. Maia, D. Kelley, Z. Erkin, M. De Cock, and A. Nascimento, "Privacy-preserving membership queries for federated anomaly detection," *Proceedings on Privacy Enhancing Technologies*, 2024.
- [58] M. Ben-Or, S. Goldwasser, and A. Wigderson, "Completeness theorems for non-cryptographic fault-tolerant distributed computation," in *Providing sound foundations for cryptography: on the work of Shafi Goldwasser and Silvio Micali*, 2019, pp. 351–371.
- [59] P. Hallgren, C. Orlandi, and A. Sabelfeld, "Privatepool: Privacy-preserving ridesharing," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 276–291.
- [60] B. Kreuter, "Secure multiparty computation at google. real world crypto," 2017.
- [61] M. Yung, "From mental poker to core business: Why and how to deploy secure computation protocols?" in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1–2.
- [62] B. Pinkas, T. Schneider, O. Tkachenko, and A. Yanai, "Efficient circuit-based psi with linear communication," in *Advances in Cryptology—EUROCRYPT 2019: 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19–23, 2019, Proceedings, Part III 38*. Springer, 2019, pp. 122–153.
- [63] Y. Yang, X. Dong, Z. Cao, J. Shen, R. Li, Y. Yang, and S. Dou, "Empsi: Efficient multiparty private set intersection (with cardinality)," *Frontiers of Computer Science*, vol. 18, no. 1, p. 181804, 2024.
- [64] L. Kissner and D. Song, "Privacy-preserving set operations," in *Annual International Cryptology Conference*. Springer, 2005, pp. 241–257.
- [65] J. Gao, S. Nguyen, and N. Trieu, "Toward a practical multi-party private set union," *Cryptology ePrint Archive*, 2023.
- [66] C. Zhang, Y. Chen, W. Liu, M. Zhang, and D. Lin, "Linear private set union from {Multi-Query} reverse private membership test," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 337–354.
- [67] J. Vos, M. Conti, and Z. Erkin, "Fast multi-party private set operations in the star topology from secure ands and ors," *Cryptology ePrint Archive*, 2022.
- [68] X. Liu and Y. Gao, "Scalable multi-party private set union from multi-query secret-shared private membership test," in *International conference on the theory and application of cryptography and information security*. Springer, 2023, pp. 237–271.
- [69] F. Kerschbaum, "Outsourced private set intersection using homomorphic encryption," in *Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security*, 2012, pp. 85–86.
- [70] T. Duong, D. H. Phan, and N. Trieu, "Catalic: Delegated psi cardinality with applications to contact tracing," in *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 2020, pp. 870–899.
- [71] A. Abadi, S. Terzis, and C. Dong, "Feather: Lightweight multiparty updatable delegated private set intersection," *Cryptology ePrint Archive*, 2020.
- [72] Y. Yang, Y. Yang, X. Chen, X. Dong, Z. Cao, and J. Shen, "Dmpsi: Efficient scalable delegated multiparty psi and psi-ca with oblivious prf," *IEEE Transactions on Services Computing*, vol. 17, no. 2, pp. 497–508, 2024.
- [73] S. Garg, M. Hajiabadi, A. Jain, Z. Jin, O. Pandey, and S. Shiehian, "Credibility in private set membership," in *IACR International Conference on Public-Key Cryptography*. Springer, 2023, pp. 159–189.
- [74] E. Chielle, H. Gamil, and M. Maniatakos, "Real-time private membership test using homomorphic encryption," in *2021 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2021, pp. 1282–1287.
- [75] S. Ramezani, T. Meskanen, M. Naderpour, V. Junnila, and V. Niemi, "Private membership test protocol with low communication complexity," *Digital Communications and Networks*, vol. 6, no. 3, pp. 321–332, 2020.
- [76] A. Kulshrestha and J. Mayer, "Identifying harmful media in {End-to-End} encrypted communication: Efficient private membership computation," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 893–910.
- [77] K. C. Wang and M. K. Reiter, "How to end password reuse on the web," *arXiv preprint arXiv:1805.00566*, 2018.
- [78] Y. Chen, M. Zhang, C. Zhang, M. Dong, and W. Liu, "Private set operations from multi-query reverse private membership test," in *IACR international conference on public-key cryptography*. Springer, 2024, pp. 387–416.
- [79] K. Chida, K. Hamada, A. Ichikawa, M. Kii, and J. Tomida, "Communication-efficient inner product private join and compute with cardinality," in *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, 2023, pp. 678–688.
- [80] K. Han, S. Kim, and Y. Son, "Private computation on common fuzzy records," *Proceedings on Privacy Enhancing Technologies*, 2025.
- [81] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, 2009.
- [82] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," *Journal of the ACM (JACM)*, vol. 60, no. 6, pp. 1–35, 2013.

- [83] C. Gentry, *A fully homomorphic encryption scheme*. Stanford university, 2009.
- [84] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, “(leveled) fully homomorphic encryption without bootstrapping,” *ACM Transactions on Computation Theory (TOCT)*, vol. 6, no. 3, pp. 1–36, 2014.
- [85] Z. Brakerski, “Fully homomorphic encryption without modulus switching from classical gapsvp,” in *Advances in Cryptology—CRYPTO 2012: 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2012. Proceedings*. Springer, 2012, pp. 868–886.
- [86] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *Cryptology ePrint Archive*, 2012.
- [87] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Tfhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [88] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. Rasmussen, and A. Sahai, “Threshold cryptosystems from threshold fully homomorphic encryption,” in *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part I* 38. Springer, 2018, pp. 565–596.
- [89] G. Asharov, A. Jain, A. López-Alt, E. Tromer, V. Vaikuntanathan, and D. Wichs, “Multiparty computation with low communication, computation and interaction via threshold fhe,” in *Advances in Cryptology—EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15–19, 2012. Proceedings* 31. Springer, 2012, pp. 483–501.
- [90] E. Kim, J. Jeong, H. Yoon, Y. Kim, J. Cho, and J. H. Cheon, “How to securely collaborate on data: Decentralized threshold he and secure key update,” *IEEE Access*, vol. 8, pp. 191 319–191 329, 2020.
- [91] S. Badrinarayanan, P. Miao, S. Raghuraman, and P. Rindal, “Multiparty threshold private set intersection with sublinear communication,” Springer, 2021, pp. 349–379.
- [92] G. Jiang, H. Zhang, J. Lin, F. Kong, and L. Yu, “Optimized verifiable delegated private set intersection on outsourced private datasets,” *Computers & Security*, vol. 141, p. 103822, 2024.
- [93] K. EdalatNejad, M. Raynal, W. Lueks, and C. Troncoso, “Private collection matching protocols,” *arXiv preprint arXiv:2206.07009*, 2022.
- [94] R. A. Popa, F. H. Li, and N. Zeldovich, “An ideal-security protocol for order-preserving encoding,” in *2013 IEEE symposium on security and privacy*. IEEE, 2013, pp. 463–477.
- [95] D. Natarajan, A. Loveless, W. Dai, and R. Dreslinski, “Chex-mix: Combining homomorphic encryption with trusted execution environments for oblivious inference in the cloud,” in *2023 IEEE 8th European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2023, pp. 73–91.
- [96] J. H. Cheon, D. Kim, and D. Kim, “Efficient homomorphic comparison methods with optimal complexity,” in *Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II* 26. Springer, 2020, pp. 221–256.
- [97] S. Chatel, C. Knabenhans, A. Pyrgelis, C. Troncoso, and J.-P. Hubaux, “Veritas: Plaintext encoders for practical verifiable homomorphic encryption,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2520–2534.
- [98] H. Chen, K. Laine, and P. Rindal, “Fast private set intersection from homomorphic encryption,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1243–1255.
- [99] N. P. Smart and F. Vercauteren, “Fully homomorphic simd operations,” *Designs, codes and cryptography*, vol. 71, pp. 57–81, 2014.
- [100] A. R. T. Donders, G. J. Van Der Heijden, T. Stijnen, and K. G. Moons, “A gentle introduction to imputation of missing values,” *Journal of clinical epidemiology*, vol. 59, no. 10, pp. 1087–1091, 2006.
- [101] D. Vatsalan, Z. Sehili, P. Christen, and E. Rahm, “Privacy-preserving record linkage for big data: Current approaches and research challenges,” *Handbook of big data technologies*, pp. 851–895, 2017.
- [102] B. Pan, N. Stakhanova, and S. Ray, “Data provenance in security and privacy,” *ACM Computing Surveys*, vol. 55, no. 14s, pp. 1–35, 2023.
- [103] Y. Zhang, A. O’Neill, M. Sherr, and W. Zhou, “Privacy-preserving network provenance,” *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1550–1561, 2017.
- [104] M.-I. Pleša and R. F. Olimid, “Privacy-preserving multi-party search via homomorphic encryption with constant multiplicative depth,” *Cryptology ePrint Archive*, 2024.
- [105] J. Lauinger, J. Ernstberger, A. Finkenzeller, and S. Steinhorst, “Janus: Fast privacy-preserving data provenance for tls,” *Proceedings on Privacy Enhancing Technologies*, 2025.
- [106] A. A. Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Saponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, “OpenFHE: Open-source fully homomorphic encryption library,” *Cryptology ePrint Archive*, Paper 2022/915, 2022, <https://eprint.iacr.org/2022/915>. [Online]. Available: <https://eprint.iacr.org/2022/915>
- [107] M. Albrecht, M. Chase, H. Chen, J. Ding, S. Goldwasser, S. Gorbunov, S. Halevi, J. Hoffstein, K. Laine, K. Lauter, S. Lokam, D. Micciancio, D. Moody, T. Morrison, A. Sahai, and V. Vaikuntanathan, “Homomorphic encryption security standard,” HomomorphicEncryption.org, Toronto, Canada, Tech. Rep., November 2018.
- [108] IBM Editorial Team, “What is logistic regression?” <https://www.ibm.com/think/topics/logistic-regression>, 2021, accessed: 2025-03-20.
- [109] D. Das, S. Meiser, E. Mohammadi, and A. Kate, “Anonymity trilemma: Strong anonymity, low bandwidth overhead, low latency-choose two,” in *2018 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2018, pp. 108–126.

APPENDIX A PARAMETER SELECTION FOR VAFs

When constructing our improved VAFs through Theorem 1, we need careful parameter selections on both the weak DEP and bell-shaped function. In this section, we introduce our method for parameter selection through grid search and provide useful presets that were used in experiments.

Calculation of Separation Parameter: To find a suitable parameter, we first need a method to obtain the separation parameter ϵ_{sep} of weak DEPs. We show that it can be easily calculated without inspecting the whole domain. According to our wDEP definition, we can observe that $|f_{\text{wDEP}}(z)| \geq \epsilon_T$ whenever $|z| \geq T$. Hence, we can break down the range for measuring ϵ_{sep} by

$$\epsilon_{\text{sep}} = \begin{cases} \epsilon_T & \text{if } T \leq 1, \\ \min\{\epsilon_T, \epsilon_{\text{sep},T}\} & \text{otherwise} \end{cases}$$

where $\epsilon_{\text{sep},T} = \min\{|f_{\text{wDEP}}(x)| : x \in [-T, T] \cap (\mathbb{Z} - \{0\})\}$. Here, $\epsilon_{\text{sep},T}$ can be found by $2\lfloor T \rfloor$ evaluations of $f_{\text{wDEP}}(x)$. Moreover, if we use $f(x; k)$ the base function, then the resulting f_{wDEP} becomes a monotonic function for $[-T, T]$ if we select the smallest T for a fixed ϵ_T , i.e., $\epsilon_{\text{sep},T} = f_{\text{wDEP}}(1)$.

Grid Search over Parameters: We can summarize the parameters we need to tune as follows:

- Parameters for the wDEP.
 - The expansion rate (L)
 - The initial domain range (R)
 - The number of domain extensions (n_{DEP})
- Parameters for the bell-shaped function.
 - The number of transformations (n_T)
 - The number of squarings (n_{SQ})

Suppose that our goal is to design a VAF for the domain $[-M, M]$ with an error of at most ν . If we denote ϵ_{sep} and $(B, \epsilon_{\text{BS}})$ as the separation factor of the resulting wDEP and the

TABLE V: Parameter Presets for Each Domain Size

Parameter	2^1	2^2	2^4	2^5	2^6	2^8	2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
k	N/A	N/A	N/A	17	17	17	6.75	6.75	6.75	17	17	6.75
L	N/A	N/A	N/A	4	4	4	2.59	2.59	2.59	4	4	2.59
R	2	4	16	4	11	4	158.54	91.09	148.45	5112.73	73139	12583
n_{DEP}	0	0	0	2	1	3	2	4	5	2	1	5
n_{VAF}	4	7	4	3	4	4	8	7	8	16	20	16
newVAF	F	F	T	T	T	T	T	T	T	T	T	T
depth	7	10	13	16	15	19	22	25	28	32	35	38

parameter for the resulting bell-shaped function, respectively, then these parameters should satisfy the following conditions:

- 1) $L^{n_{DEP}} R \geq M$ to ensure $[-M, M] \subset [-L^{n_{DEP}} R, L^{n_{DEP}} R]$
- 2) $B \leq \nu$ to ensure sufficient accuracy.
- 3) $\epsilon_{BS} \leq \epsilon_{sep}$ to ensure the condition of Theorem 1.

From these requirements, we can derive several relationships between parameters. From the first condition, we need to ensure that $n_{DEP} > \frac{\log_2 M - \log_2 R}{\log_2 L}$, i.e., once R is chosen, then the smallest integer for n_{DEP} is determined. For the second condition, recall that the proposed bell-shaped function is constructed upon transforming the initial function $f(x) = (1 - \frac{3}{2}x^2)^2$, which is $(0.25, \sqrt{1/3})$ -bell-shaped function. Since our transformation does not change the bound parameter, the final bound B solely depends on the number of final squarings n_{SQ} , namely $2^{-2^{n_{SQ}+1}}$. That is, $2^{n_{SQ}+1} \geq -\log_2 \nu$ is sufficient for the second condition.

Analyzing the third condition looks non-trivial, but we can utilize numerical algorithms to analyze each of ϵ_{sep} and ϵ_{BS} . For the former, recall that ϵ_{sep} is strongly related to the tail bound ϵ_T and the separation parameter in a smaller domain, i.e., $\epsilon_{sep,T} = \min\{|f_{sep}(x)| : x \in [-T, T] \cap (\mathbb{Z} - \{0\})\}$. Note that $\epsilon_{sep,T} = f_{wDEP}(1)$ if the resulting function is monotonic in the range $[-T, T]$. Since our base function $D(x; k) = \frac{3\sqrt{3}}{2k\sqrt{k}}(k - x^2)$ is indeed monotonic for the interval $[-\sqrt{k/3}, \sqrt{k/3}]$, we can safely ensure the above property whenever $k \geq 3$. In addition, for the base function $D(x)$ of the f_{wDEP} , $\epsilon_T = D(L)$ holds because of the domain extension process.

On the other hand, recall that ϵ_{BS} has a strong relationship between the second derivative of f_{BS} , namely, $\epsilon_{BS} \approx \sqrt{\frac{1-B}{-f_{BS}''(0)}}$. However, this estimation involves the error from the Taylor approximation, so we need a more accurate estimation. To this end, we use another estimation of ϵ_{BS} by solving the root-finding problem $f_{BS}(x) = B$ nearby 0. We found that a simple bisection search method is enough for our purpose.

To sum up, once the required conditions, the domain $[-M, M]$ and the precision ν , and the base function for the wDEP are fixed, we can conduct a grid search on parameters by the following pipeline.

- 1) Make a grid for possible $R \in [1, M]$. Then we obtain the corresponding n_{DEP} to satisfy the condition (1).
- 2) Compute ϵ_{sep} from the given weak DEP parameters.
- 3) Choose n_{SQ} to satisfy the condition (2) and find the suitable n_T to satisfy the condition (3) through solving the root-finding algorithm $f_{BS}(x) = B$.

Once parameters are found, we select the parameter requiring the smallest multiplicative depth. If there are ties, then we select one having a smaller n_T because the transformation for the bell-shaped function is cheaper than the domain extension. Detailed algorithms for homomorphically evaluating the VAF from parameters along with the cost analysis are provided in Section C.

Useful Presets: We provide the parameters obtained from the aforementioned grid search. The presets are provided in Table V. We provide the implementation of the grid search algorithm through our anonymized source code.

APPENDIX B ADDITIONAL ANALYSES

We provide additional analyses on our choice of the base function and the κ for the slow-wise windowing technique.

Validation on the Base Function: We justify our choice of the base function $D(x; k) = \frac{3\sqrt{3}}{2k\sqrt{k}}x(k - x^2)$. Recall that we presented the following three conditions on the base function for constructing wDEPs.

- $D(x) = 0$ if and only if $x = 0$.
- $|D(x)| \leq 1, \forall x \in [-L, L]$.
- $\exists x_1, x_2 \in (-L, L)$ s.t. $D(x_1) = -1$ and $D(x_2) = 1$.

These conditions are strongly tied to the domain extension process, so we first briefly explain how domain extension works. Let $D(x)$ be a base function and suppose that we wish to extend it to $[-L^n, L^n]$. For simplicity, we consider the case when $R_1 = 1$. Then for the input $y_n \in [-L^n, L^n]$, we recursively compute $y_i \leftarrow L^i D(\frac{y_{i+1}}{L^{i+1}})$ for $i = n-1$ to 0. From this, we can observe that the input $\frac{y_{i+1}}{L^{i+1}}$ of $D(\cdot)$ always take a value between $[-L, L]$ if $D(z) \in [-1, 1]$ for any $z \in [-L, L]$. In addition, we also note that $D(\hat{z}) = \pm 1$ for some $\hat{z} \in (-L, L)$, because if this is not the case, i.e., $\max_{x \in [-L, L]} D(x) = M < 1$, then $\frac{y_{i+1}}{L^{i+1}} \in (-M^{n-i+1}L, -M^{n-i+1}L)$. This implies that the resulting values of the resulting wDEP become close to 0 for all inputs, so we may not select desirable parameters (T, ϵ_T) for the tail parts. All these requirements correspond to each condition.

We now verify that our choice of the base function satisfies all the conditions. The first condition is straightforward. To analyze the second and third ones, we can observe that the function takes extreme values at $x = \pm\sqrt{k/3}$, which can be obtained by a simple algebraic manipulation. In particular, $D(\sqrt{k/3}; k) = 1$. Finally, $D(x; k)$ monotonically decreases at $x > \sqrt{k/3}$ and $D(\sqrt{k}; k) = 0$. From this information, we can

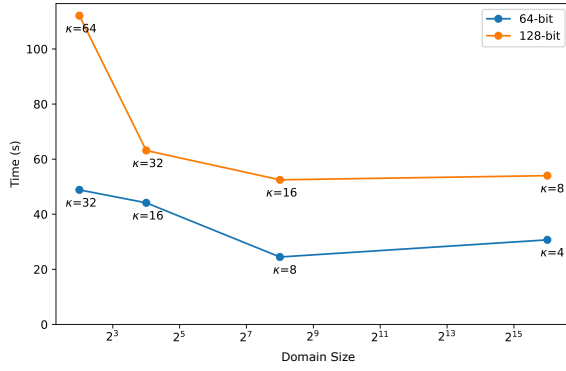


Fig. 8: Computation time of different κ values for the VLDP dataset

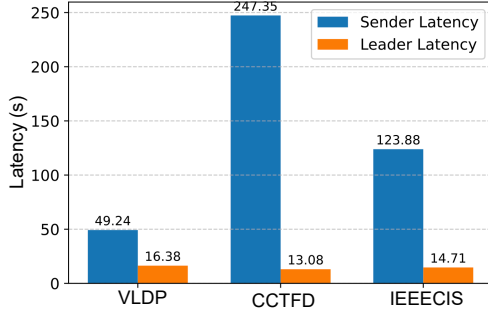


Fig. 9: Runtime latency for ELSA on 3 FDB datasets. $\sigma = 64$ and $\kappa = 8$.

conclude that if we set $L \in (\sqrt{k/3}, \sqrt{k})$, then the proposed base function satisfies all these conditions.

Selecting κ for Long Items in Slot-Wise Windowing: We provide a concrete algorithm for applying slot-wise windowing for $\sigma = 64$ and $\sigma = 128$ in Algorithm 4. κ is inversely proportional to the required approximation parameters; however, it is directly proportional to the number of ciphertexts required to store the set elements. Choosing a small κ can significantly increase the required approximation parameters, leading to increased computation and communication overhead, while choosing a very large κ can dramatically increase the number of ciphertexts required to store the set elements, thus increasing memory overhead. We found that choosing a moderately large value of κ , about 7 to 10, can lead to significant performance improvements in the protocol. We observe this performance increase because the κ ciphertexts required to store the set elements are completely independent from each other and can be processed in a parallel manner. We note that one also needs to take into account the type of application before choosing κ . Computation latencies of using various κ values for the VLDP dataset are shown Figure 8. We report the total computational latency for all 3 FDB datasets in a single server setting in Figure 9. Some medical applications that require a smaller input domain but higher precision might require relatively smaller κ values to maintain higher precision during approximation.

Algorithm 1 Depth-Efficient Transformation DETBS

Require: Input $x \in \mathbb{R}$ and the number of transformations $n \in \mathbb{N}$, and transformation parameters $\frac{a}{q}, \frac{b}{q} \in \mathbb{Q}$

Ensure: Result y from applying $x \mapsto \frac{a}{q}x + \frac{b}{q}$ n times.

- 1: Initialize $Q \leftarrow 1$.
- 2: **for** idx from 1 to $n - 1$ **do**
- 3: Compute $x \leftarrow (ax + Qb)^2$.
- 4: Update $Q \leftarrow Q^2 \cdot q^2$.
- 5: **end for**
- 6: Compute $x \leftarrow \frac{1}{Q \cdot q}(ax + Qb)$ and $y \leftarrow x^2$.
- 7: **return** y .

APPENDIX C ALGORITHMS

Fused DEP Iteration: We provide detailed algorithms for improved VAFs introduced in Section VI-A. We first provide the algorithm of the depth-efficient transformation technique, which is provided in Algorithm 1, called DETBS.

Theoretically, we can use algorithm DETBS to design the whole VAF. However, we can observe that the intermediate value in x exponentially grows with respect to the number of iterations n , which may result in overflow during evaluation. For this reason, we restrict the maximum number of fused transformations per one DETBS, say $n_{\max}$. When evaluating the VAF with transformations $n > n_{\max}$, we first divide n by $n_{\max}$ to get the quotient and remainder, say (q, r) . Then we run DETBS with $n_{\max}$ q times and after evaluation, we run DETBS with r on the output. We describe the above process in Algorithm 2, called VAFfromBS.

Finally, we merge the VAFfromBS algorithm to the domain extension process, with some tricks to reduce the depth. First, we slightly tweak the order of computation when evaluating the base function $D(x; k) = \frac{3\sqrt{3}}{2k\sqrt{k}}x(k - x^2)$, namely,

$$y_1 \leftarrow k - x^2; \quad y_2 \leftarrow \frac{3\sqrt{3}}{2k\sqrt{k}}x; \quad y \leftarrow y_1 \cdot y_2.$$

In particular, we can save more levels at the domain extension process, i.e., $y \mapsto L^i D(\frac{y}{L^i})$ because we can think of $\frac{y}{L^i}$ as $\frac{1}{L^i} \cdot L^{i+1} D(\frac{\hat{y}}{L^{i+1}})$, where $\hat{y}$ is the input of previous domain extension processes. That is, we can tweak the domain extension process as $y \leftarrow LD(y)$ with some careful scaling at the beginning. In addition, multiplication by L at the end can be applied by letting $y_2 \leftarrow \frac{3\sqrt{3}L}{2k\sqrt{k}}x$. Hence, we can compute the domain extension process using 2 levels per iteration.

In addition, we also observed that when we combine the weak DEP and our VAF, we can save one level when calculating $(1 - \frac{3}{2}x^2)^2$ by multiplying $\sqrt{3}/2$ to y_2 at the last domain extension process. We describe the complete VAF construction in Algorithm 3, which was used for our protocol.

Slot-Wise Windowing: The slot-wise windowing enables us to construct a VAF over larger domain using ones over smaller domains. We provide the algorithm to split each set item to chunks through our windowing technique in Algorithm 4. Later, at the VAF evaluation phase, we can compute the VAF

Algorithm 2 VAF from Bell-Shaped Function (VAFfromBS)

Require: Input $x \in \mathbb{R}$, the number of transformations $n \in \mathbb{N}$, transformation parameters $\frac{a}{q}, \frac{b}{q} \in \mathbb{Q}$, and the maximum number of fused transformations $n_{\max}$.

Ensure: Result y from applying $x \mapsto \frac{a}{q}x + \frac{b}{q}$ n times.

- 1: Compute $q, r \in \mathbb{N}$ such that $n = q \cdot n_{\max} + r$ and $0 \leq r < n_{\max}$.
- 2: **for** idx from 1 to q **do**
- 3: Compute $x \leftarrow \text{DETBS}(x, n_{\max}, \frac{a}{q}, \frac{b}{q})$
- 4: **end for**
- 5: Compute $y \leftarrow \text{DETBS}(x, r, \frac{a}{q}, \frac{b}{q})$.
- 6: **return** y .

Algorithm 3 VAF from Weak DEP and Bell-Shaped Functions

Require: Input $x \in \mathbb{R}$, wDEP parameter k , number of extensions $n_{\text{DEP}} \in \mathbb{N}$, extension rate L , base domain range R , and $n_T, n_{\text{SQ}} \in \mathbb{N}$ for the number of transformations and squarings for bell-shaped function.

Ensure: The VAF evaluation result $f^{(n)}(x; k)$.

- 1: Set $y \leftarrow \frac{x}{L^{n-1}R}$.
- 2: **for** idx from 1 to n_{DEP} **do**
- 3: Compute $\tilde{y} \leftarrow (k - y^2)$
- 4: If $\text{idx} < n_{\text{DEP}}$, then set $y \leftarrow \frac{3\sqrt{3}}{2k\sqrt{k}}Ly$; else, $y \leftarrow \frac{9}{2\sqrt{2}k\sqrt{k}}y$.
- 5: Compute $y \leftarrow y \cdot \tilde{y}$.
- 6: **end for**
- 7: Set $y \leftarrow (1 - y^2)^2$
- 8: $y \leftarrow \text{VAFfromBS}(y, n_T, \frac{3}{2}, -\frac{1}{2})$:
- 9: **for** idx from 1 to n_{SQ} **do**
- 10: Compute $y \leftarrow y^2$
- 11: **end for**
- 12: **return** y

for encoded with slot-wise windowing by (1) computing the VAF in parallel across slots, and (2) apply the rotation-and-multiplication to multiplicatively aggregate the results in the slots. To prevent information leakage of set items from partial matching, we need to multiply a masking vector that allows us to extract values in slots of stride κ .

APPENDIX D

DATASETS

Below, we provide a detailed description of the datasets used in the functional computation evaluation of our protocol.

- 1) IEEE-CIS Fraud Detection: It contains 590K card-not-present transactions with 67 features. We processed the dataset to obtain 25 features with proper encoding for CKKS plaintexts.
- 2) Sparkov (Simulated) Credit Transactions: This dataset contains about 1.2 million entries and it covers transactions of 1000 customers with a pool of about 800 merchants over 6 months. We train a small model on 25 relevant features after processing the dataset. This dataset highlights the large scale and demonstrates how our protocol scales

Algorithm 4 SLOT_WISE_WINDOWING

Require: Bit length $\sigma \in \{64, 128\}$; windowing count $\kappa \in \mathbb{N}$; integer element e either in $[0, 2^\sigma - 1]$ or in $[-2^{\sigma-1}, 2^{\sigma-1} - 1]$.

Ensure: An array Windows $= \{c_1, \dots, c_\kappa\}$, where each c_i is in the domain $[-2^{\lfloor \sigma/\kappa \rfloor - 1}, 2^{\lfloor \sigma/\kappa \rfloor - 1}]$.

- 1: $\text{exponent} \leftarrow \lfloor \sigma/\kappa \rfloor$
- 2: $\text{domain} \leftarrow 2^{\text{exponent}}$
- 3: Initialize array Windows of length κ
- 4: **for** $i \leftarrow 1$ to κ **do**
- 5: $c_i \leftarrow e \bmod \text{domain}$ // Extract lower exponent bits
- 6: $e \leftarrow \lfloor e/\text{domain} \rfloor$ // Reduce σ -bit integer by exponent bits
- 7: **if** $c_i > \frac{\text{domain}}{2}$ **then**
- 8: $c_i \leftarrow c_i - \text{domain}$ // Optional: map to negative range
- 9: **end if**
- 10: Windows[i] $\leftarrow c_i$
- 11: **end for**
- 12: **return** Windows

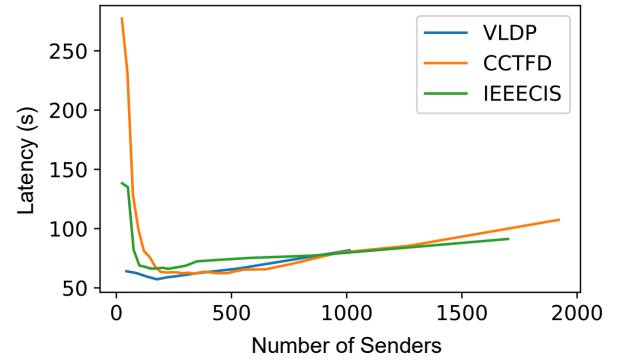


Fig. 10: Total computational latency for ELSA on 3 FDB datasets for an increasing number of senders. $\sigma = 64$ and $\kappa = 8$.

beyond the 1M data limit or for multi-senders with tens of millions of rows.

- 3) Vehicle Loan Default: The task in this dataset is to determine the probability of vehicle loan default. It contains data for 233K loans with 21.7% default rate and 44 features after pre-processing.

Omitted Experimental Data for the Datasets

Table VI shows the total latency per server and communication cost between the parties for three FDB datasets under different threads. Since the number of senders is doubled in the $\delta = 128$ bit setting, the latency of the senders remains almost the same as each sender has to process fewer ciphertext or records. Figure 10 shows the effect of increasing number of senders on the total computational latency of ELSA for the FDB datasets.

TABLE VI: ELSA latency results for FDB datasets (per query). Total computation times and communication cost between sender(s) and receiver. T denotes the number of threads. Sender count and storage refer to the number of senders and storage needed for each sender, respectively. $\kappa = 8$ for $\delta = 64$ and $\kappa = 16$ for $\delta = 128$.

Item Length (bits)	Dataset	Sender		Online Comm. (MB)			Online Latency (s)		
		Count	Storage(MB)	R→S	S→L	S→R	T=8	T=16	T=48
64 ($\kappa=8$)	VLDP	176	1291.8	92.3	39.8	16.8	62.6	63.2	58.4
	CCTFD	960	738.2	92.3	39.8	16.8	63.8	69.8	79.5
	IEECIS	250	1291.9	92.3	39.8	16.8	58.9	62.9	66.9
128 ($\kappa=16$)	VLDP	308	1321.2	92.3	39.8	16.8	60.6	63.4	67.2
	CCTFD	1920	755.0	92.3	39.8	16.8	74.6	78.3	112.0
	IEECIS	500	1321.21	92.3	39.8	16.8	62.8	64.2	74.2

APPENDIX E FORMAL SECURITY PROOFS

We present the security proofs of the proposed protocol, which was omitted in the main text.

Security Proof of the Proposed Protocol

We first show the security of our protocol under the semi-honest adversary model. For simplicity, we denote the proposed protocol as π_{ELSA} .

To this end, we introduce security notions of the threshFHE needed for our security proof. First, we need the semantic security of the threshFHE, which tells us that any party having secret key shares less than the decryption threshold cannot distinguish the given threshFHE ciphertext from a random string. In addition, we also introduce the notion of simulation security of threshFHE, which guarantees the existence of a simulator satisfying the following properties. The simulator takes the circuit, the ciphertexts of the input and outputs, and the plaintext of the evaluation result as inputs, and returns partial decryptions of the output ciphertext from some parties' secret key shares that are (computationally) indistinguishable from the real protocol execution. For more information, we recommend the reader refer to Definition 5.4 & Definition 5.5 in [88] or Section 2.5 in [91].

Equipped with these definitions, we will prove the following statement.

Theorem 2. *Assuming that the underlying threshold FHE scheme used satisfies semantic security and simulation security, then π_{ELSA} is a secure implementation of the ideal functionality of Π_{ELSA} as described in Figure 2 according to Definition 1.*

Proof. To complete the proof, we must show that there exists a simulator for the four different types of adversaries possible:

- 1) A number of the senders $\{S_n\}$ are corrupted, excluding the leader and receiver.
- 2) A number of senders are corrupted $\{S_{n-1}\}$, including the leader S_n , but the receiver is not corrupted.
- 3) The receiver $\mathcal{R}$ is corrupted along with a subset of the senders $\{S_n\}$, which does not include the leader sender.
- 4) The receiver $\mathcal{R}$ is corrupted along with a subset of senders $\{S_{n-1}\}$ which does include the leader sender S_n .

We will construct the simulator for each case. Since we assumed that the setup phase is implemented by an SMPC

protocol or trusted setup, and the data owner does not join the protocol after outsourcing, we focus on simulating the views for the querying and retrieving phase. For this reason, the simulator can obtain the public keys, including the encryption key and evaluation key, of the threshold FHE, and the secret key shares of corrupted parties.

Case 1.: In this case, the adversary is assumed not to have corrupted the receiver $\mathcal{R}$ or the leader sender. As such, the simulator $\mathcal{P}$ can be defined as follows: $\mathcal{P}$ simulates the view of the corrupted sender parties $\{S_n\}$ by taking the input $\{\mathcal{X}_i\}$, i.e., the encrypted sets provided by each data owner corresponding to the corrupted parties. Then, $\mathcal{P}$ arbitrarily generated ciphertext c'_y , and then sends it to the corrupted parties. Finally, $\mathcal{P}$ concludes the protocol by playing the role of both the leader sender and the receiver by distributing an arbitrarily generated ciphertext pair $(z_{\text{sim}}, \overline{\text{flag}_{\text{sim}}})$ with setting appropriate levels for them through modulus reduction, and accepting the resultant partial decryptions from some random selection of the dishonest parties.

By the assumed security of the threshold FHE scheme, any encrypted ciphertext will be indistinguishable. Therefore, $\mathcal{P}$ can encrypt an arbitrary number and perform the computations to generate a fake c'_y and $(z_{\text{sim}}, \overline{\text{flag}_{\text{sim}}})$ sent to the senders which can be indistinguishable from a true c_y and $(z, \overline{\text{flag}})$.

Case 2.: In this case, the adversary here is assumed to have corrupted some senders in $S_{1,\dots,n-1}$ and the leader sender denoted here as S_n , but not the receiver. Here the simulator $\mathcal{P}$ can be defined similarly to case 1: As in Case 1, $\mathcal{P}$ simulates the view of corrupted sender parties by taking some encrypted sets $\{\mathcal{X}_i\}$ from the corrupted as inputs. During the Encryption step, $\mathcal{P}$ randomly generates an arbitrary ciphertext c'_y and sends the transmission to the corrupted parties. To simulate the Label Extraction, $\mathcal{P}$ sends arbitrarily generated ciphertexts $(\text{resLab}_{\text{sim}}, \text{flag}_{\text{sim}})$ with setting appropriate levels for them by acting as the non-corrupted senders. Once the leader sender sends the information from the Computation step to $\mathcal{P}$, $\mathcal{P}$ collects the resultant partial decryptions before concluding the protocol.

As in the first case, due to the assumed security of the threshold FHE scheme, any encrypted ciphertext will be indistinguishable. Therefore, $\mathcal{P}$ can send a simulated ciphertext c'_y to the corrupted parties. In a similar fashion, the $\mathcal{P}$ is also able to create arbitrary ciphertexts to generate fake $\{\text{resLab}_{\text{sim}}, \text{flag}_{\text{sim}}\}$ values which are sent to the

leader sender, indistinguishable from a true collection of $\{resLab_i, flag_{sim}\}$ values.

Case 3.: In this case, along with the receiver $\mathcal{R}$, the senders $\{\mathcal{S}_n\}$ are assumed to be corrupted, which does not include the leader sender. Here, the simulator $\mathcal{P}$ takes the query y from the receiver, the function evaluation result f_{eval} , the flag bit $b \in \{0, 1\}$, and the encrypted set $\{\mathcal{X}_i\}$ held by corrupted senders. The simulator crafts the views of the corrupted parties by the following procedure.

1. By using the flag bit b , which tells us whether $y \in \{\mathcal{X}_i\}$, $\mathcal{P}$ creates a simulated flag ciphertext. $flag_{sim} \leftarrow \text{thresFHE.Enc}(pk, \vec{b})$, where $\vec{b}$ is a plaintext vector whose all slots are filled by b . $\mathcal{P}$ also applies modulus reduction to set an appropriate level.

2. By using the function evaluation result f_{eval} , $\mathcal{P}$ simulates the ciphertext $z_{sim} \leftarrow \text{thresFHE.Encrypt}(pk, f_{eval})$ with setting an appropriate level.

3. In order to simulate the partial decryptions of $(z_{sim}, flag_{sim})$, $\mathcal{P}$ randomly selects a number i of corrupted senders and the receiver to distribute $(z_{sim}, flag_{sim})$. The corrupted senders will generate partial decryptions $\{(p_i, q_i)\}$. For the remaining $\alpha - i - 1$ uncorrupted senders, $\mathcal{P}$ randomly generates $\alpha - i - 2$ random ciphertexts denoted $\{(p_{sim}, q_{sim})\}$ and chooses the final ciphertext pair (p_1, q_1) to ensure the correctness of the final decryption. That is to say, if $y \in \{\mathcal{X}_i\}$, $\mathcal{P}$ chooses p_1 to ensure $f_{eval} = \text{thresFHE.Combine}(pk, \{p_i\} \cup \{p_{sim}\} \cup p_1 \cup p_{\mathcal{R}})$ where $p_{\mathcal{R}}$ is the receiver's partial decryption of z_{sim} . Otherwise, p_1 can be a generated ciphertext. For the partial decryptions of $flag_{sim}$, $\mathcal{P}$ chooses q_1 to ensure $\vec{b} = \text{thresFHE.Combine}(pk, \{q_i\} \cup \{q_{sim}\} \cup q_1 \cup q_{\mathcal{R}})$ where $q_{\mathcal{R}}$ is the receiver's partial decryption of $flag_{sim}$.

Due to the correctness of the thresFHE scheme used, the receiver will receive the correct answer when decryption the simulated $(z_{sim}, flag_{sim})$, but find the simulated partial decryptions indistinguishable from real partial decryptions.

Case 4.: In this case, the receiver $\mathcal{R}$ and the senders $\mathcal{S}_{1, \dots, n-1}$ are assumed to be corrupted as well as the leader sender $\mathcal{S}_n$. Here, the simulator $\mathcal{P}$ takes the query y from the receiver, the function evaluation result f_{eval} , the flag bit $b \in \{0, 1\}$, and the encrypted set $\{\mathcal{X}_i\}$ held by corrupted senders, and then pretends the behavior of uncorrupted sender parties as follows:

1. For the corrupted leader sender to compute the intersection, $\mathcal{P}$ sends random ciphertexts to serve as $(resLab_i, flag_i)$ results to the leader sender.

2. Once the leader sender sends $(z, flag)$ to $\mathcal{P}$, $\mathcal{P}$ computes a simulated $(z_{sim}, flag_{sim})$ as in Case 3 for the correct outputs $(f_{eval}, \vec{b})$.

When the receiver proceeds with the Combine and Output step, the protocol concludes. Since we have at most $\alpha - 1$ colluding parties, there must be at least one non-colluding party that is chosen during the partial decryption step. As such, any and all ciphertexts provided by $\mathcal{P}$ must be indistinguishable according to the underlying thresFHE scheme. By sending a share that will force the resulting decryption to give the correct answer, the simulated view of the corrupted receiver will be

the same as a real-life view. In the case that the leader sender is corrupt, because there can only be $\alpha - 1$ colluding parties, there must be at least one party that is called for their share which can force the decryption of $(z, flag)$ is identical to $(f_{eval}, \vec{b})$.

As in all cases, the simulator $\mathcal{P}$ can simulate the behavior of protocol π_{ELSA} for some number of colluding parties up to $(\alpha - 1)$, the protocol must fulfill the security definition for Π_{ELSA} protocols. $\square$

Proving Provenance Security in the Proposed Protocol

Informally, we consider a distinguishing game where the adversary (colluding parties) attempts to distinguish two protocols through transcripts, i.e., view of colluding parties: the *normal* execution of the protocol and the *permuted* protocol when the inputs are randomly permuted across senders but the remaining procedures are the same. If the adversary cannot distinguish these two cases, then we can ensure that colluding parties cannot learn *which* party contributed *which* data. Provenance security intersects with anonymity and unlinkability goals known in other domains and has direct parallels in anonymous communication systems [109]. We provide the formal definition for provenance security in Definition 4.

Definition 4 (Provenance Security). *Let Π be a n -ary protocol for a receiver $\mathcal{R}$ and senders $\mathcal{S}_1, \dots, \mathcal{S}_{n-1}$ and $\Pi_{\text{Perm}, \delta}$ be a protocol defined by randomly permuting the inputs of senders through the permutation δ over indices $[n - 1]$. Then we say that Π satisfies provenance security under the semi-honest model with at most $(\alpha - 1)$ collusion if the following inequality holds: For all PPT adversary $\mathcal{A}$ and colluding parties $\mathcal{P}^* \subset \{\mathcal{R}, \mathcal{S}_1, \dots, \mathcal{S}_{n-1}\}$ such that $|\mathcal{P}^*| \leq \alpha - 1$,*

$$\left| \Pr[\mathcal{A}(tr) = 1 \mid tr \leftarrow \text{View}_{\Pi}(\mathcal{P}^*)] - \Pr\left[\mathcal{A}(tr) = 1 \mid \begin{array}{c} \sigma \xleftarrow{\$} \text{Perm}_{n-1}, \\ tr \leftarrow \text{View}_{\Pi_{\text{Perm}, \sigma}}(\mathcal{P}^*) \end{array} \right] \right| < \text{negl}(\lambda),$$

where Perm_{n-1} is the set of all permutations over $[n - 1]$, $\text{View}_{\Pi}(\mathcal{S})$ is the union of the transcripts received by parties $\mathcal{P} \in \mathcal{S}$ during the execution of Π , and λ is the security parameter.

We prove that our protocol satisfies provenance security. Since we are using (α, l) -thresFHE to ensure the privacy of outsourced sets from the data owner, we restrict the number of the colluding parties to at most $(\alpha - 1)$. As we discussed in Section VII, achieving the provenance security for generic asymmetric functions would be inherently impossible. In addition, when considering the label retrieval scenario, i.e., the function f we want to evaluate is the identity function, each label held by senders must be of the same type, otherwise the receiver can guess the provenance through the type of the retrieved label. For this reason, we only consider the symmetric function and the identity function, and for the latter case, we require that all the labels have the same type. We provide a proof of the following statement.

Theorem 3. *Let the function f be either symmetric or the identity function. Then π_{ELSA} for evaluating f satisfies provenance security as defined in Definition 4 with at most $(\alpha - 1)$ collusions.*

Proof. Given the definition of provenance security, there are two cases to consider:

- 1) The receiver $\mathcal{R}$ is not included in $\mathcal{P}^*$
- 2) The receiver $\mathcal{R}$ is included in $\mathcal{P}^*$

These cases can be handled separately as they are mutually exclusive and their union covers all possible scenarios.

Case 1.: In the case that the adversary has not corrupted the receiver $\mathcal{R}$, then the view of the adversary is restricted to the view of the senders and possibly the leader sender. As such, the view of the adversary only includes ciphertexts and does not include the output or original input. Because the number of colluding parties cannot be larger than $\alpha - 1$, by the correctness of the ThresFHE scheme, $\mathcal{A}$ will not have access to the decryption of any of the ciphertexts within either view. That is to say, for either view, for any element $c \in \text{View}_{\Pi}(\mathcal{P}^*), \text{View}_{\Pi_{\text{perm},\rho}}(\mathcal{P}^*)$ it must be true that c is an element of the ciphertext space of the ThreshFHE scheme. Given the setup of the thresFHE scheme is the same for both views, the ciphertext space of both views will be identical. As the ThresFHE scheme satisfies semantic security, any of ciphertexts are indistinguishable from any other ciphertext within the ciphertext space. So, the ciphertexts of permuted inputs will be indistinguishable from non-permuted inputs. Thus, the view of any permutation of inputs will be indistinguishable from any other permutation of inputs, and provenance security is satisfied in this case.

Case 2.: In the case of the receiver $\mathcal{R}$ colluding with $\mathcal{A}$, the same method as in Case 1 can be used to show that all ciphertexts contained within $\text{View}_{\Pi}(\mathcal{P}^*)$ are indistinguishable from those in $\text{View}_{\Pi_{\text{perm},\rho}}(\mathcal{P}^*)$. The elements that remain in $\text{View}_{\Pi}(\mathcal{P}^*)$ that are not contained within the ciphertext space of the ThresFHE scheme are the decrypted outputs (flag, z) .

As the flag element is a boolean used to show the presence of the element in the sets of the data owners, the value of flag will not vary from $\text{View}_{\Pi}(\mathcal{P}^*)$ to $\text{View}_{\Pi_{\text{perm},\sigma}}(\mathcal{P}^*)$. Thus, the flag does not reveal any information regarding the view.

The final element to consider in the views is the decrypted output $z = f_{\text{eval}}$. If the function f is symmetric, then we have nothing to prove because the permutation on the inputs of f does not change the output, i.e., the view from with and without permutation is identical. Let us assume that the function f is the identity function, which is asymmetric. The effect of the permutation ρ by the sender when constructing the resLab values is that the input of the sender S_i is associated with the slot ρ_i as opposed to the corresponding slot i . As such, it is equivalent to performing the permutation of inputs such that the sender S_{ρ_i} 's input is present in the corresponding slot. Since each sender receives the permutation in encrypted form at the setup phase, by the semantic security of the underlying thresFHE, the colluding parties cannot get any information about the ciphertexts corresponding to permutations.

Therefore, given that the permutation of slots ρ computed by the protocol during the setup is random and the random permutation σ of ρ is drawn from the same distribution, they will have an equal probability of being any permutation p from the set of possible permutations over $[n - 1]$ even when we consider the conditional probability in terms of the view of colluded parties. Note that we assumed the labels have the same type, so we can ensure that the permutation of labels itself would not leak provenance. Therefore, since the probability of the resultant permutations is the same, the view of corrupted parties with and without permutation must be indistinguishable.

In all cases, π_{ELSA} satisfies provenance security, and therefore, the protocol must satisfy provenance security. $\square$